



8. 基础2——期中复习

授课教师：游伟 副教授、孙亚辉 副教授

授课时间：周二14:00 – 15:30，周四10:00 – 11:30（教学三楼3304）

上机时间：周二18:00 – 21:00（理工配楼二层机房）

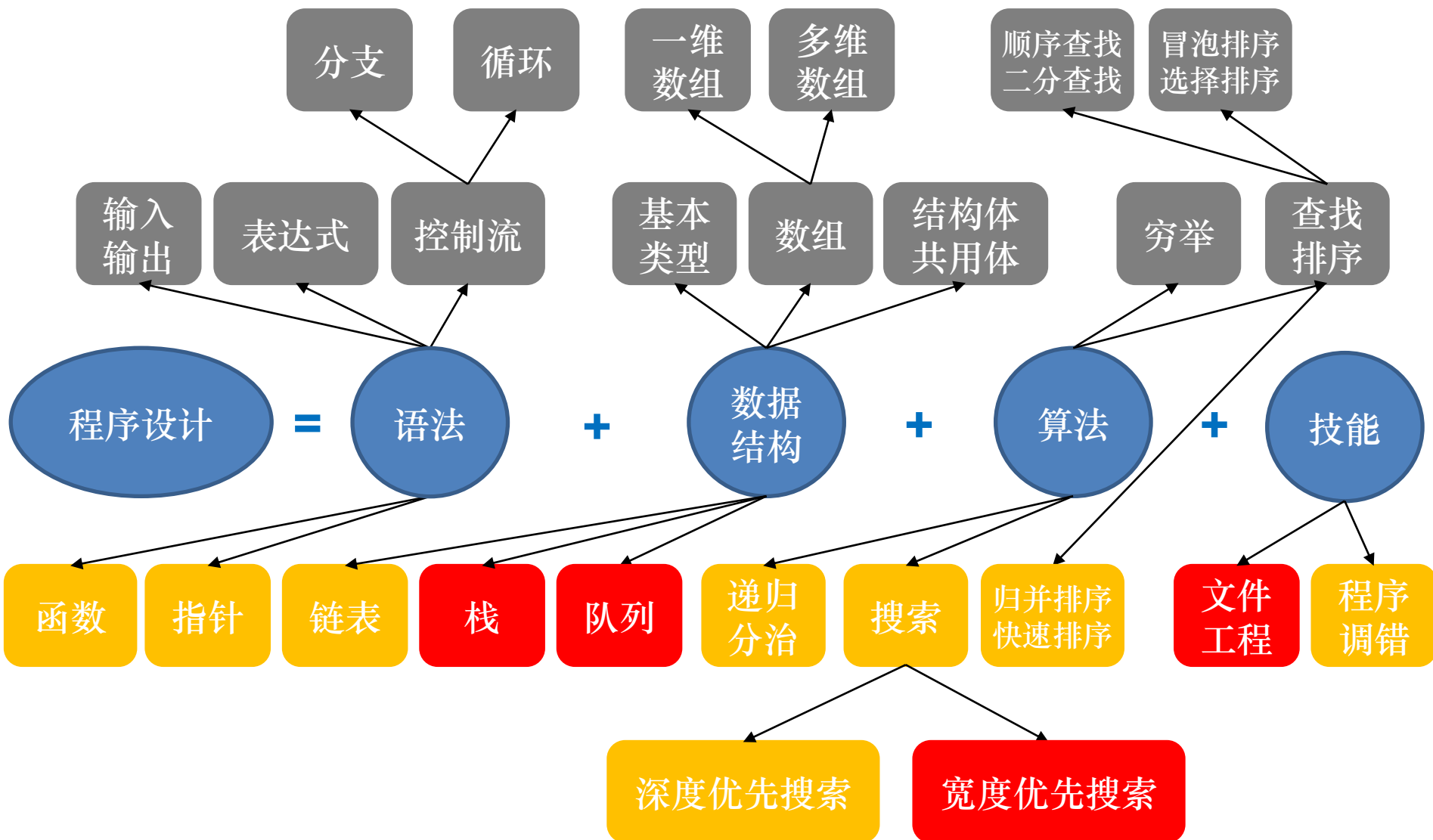
课程主页：<https://www.youwei.site/course/programming>

已学内容

后半学期
基础内容

后半学期
提高内容

知识脉络



目录

1. 基础知识回顾
2. 基本功能代码
3. 例题讲解

8.1 基础知识回顾

- 基础1——计算机与程序运行
- 语法1——程序设计基本概念
- 语法2——程序控制语句
- 算法1——穷举法
- 数据结构1——数组与自定义结构
- 算法2——查找与排序

8.1.1 基础1——计算机与程序运行

■ 计算模型与体系结构

- 图灵机模型 (TM)
- 随机存取模型 (RAM)

TM v.s. RAM

- TM的输入输出及中间结果全在一条纸带上, RAM有独立输入带、输出带, 中间结果存放于寄存器组中
- TM的中间结果只能按顺序访问, RAM上可以随机访问中间结果
- TM的规则相当于RAM的程序

■ 示例: 用TM模型计算 $x+1$

- 编码: 用二进制表示 x , 用*与符号带上的其它符号分隔开
- 符号集: 0, 1, *
- 状态集: start (初始), inc (增加), noncarry (不进位), carry (进位), overflow (溢出), halt (停止)

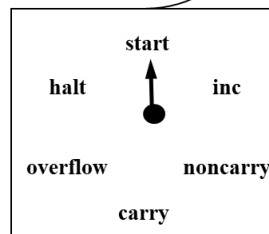
规则

当前格局	采取的动作
(start, *)	(inc, *, L)
(inc, 0)	(noncarry, 1, L)
(inc, 1)	(carry, 0, L)
(inc, *)	(halt, *, -)
(noncarry, 0)	(noncarry, 0, L)
(noncarry, 1)	(noncarry, 1, L)
(noncarry, *)	(halt, *, -)
(carry, 0)	(nocarry, 1, L)
(carry, 1)	(carry, 0, L)
(carry, *)	(overflow, 1, L)
(overflow, 0/1/*)	(halt, *, -)

纸带



读写头



状态控制器

示例: 计算 $5+1$

RAM指令集

- Input: 输入数据
- Output: 输出数据
- Load: 将普通寄存器中的值载入累加器
- Store: 将累加器中的值存入普通寄存器
- Add: 将指定寄存器的值与累加器的值相加, 结果存入累加器

■ 示例: 用RAM模型计算 $x+y$

```
1. Input R1
2. Input R2
3. Load R1      //R0 = R1
4. Add R2        //R0 = R0 + R2
5. Store R3      //R3 = R0
6. Output R3
```

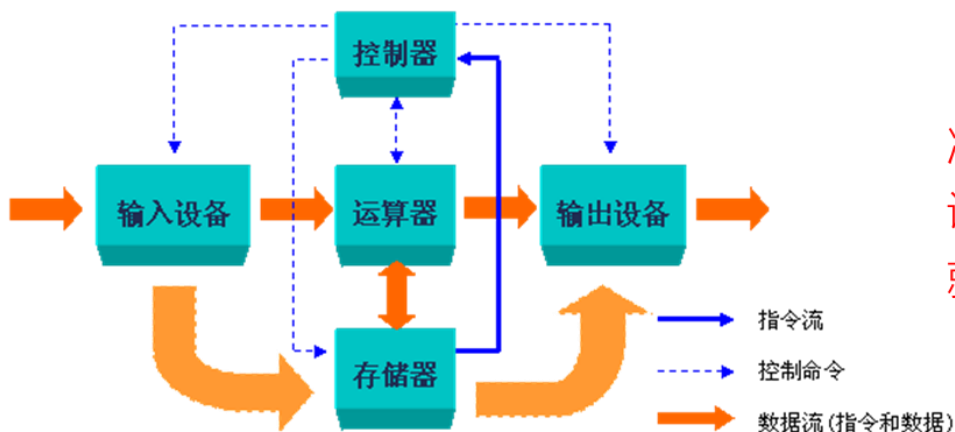
8.1.1 基础1——计算机与程序运行

■ 计算模型与体系结构

■ 体系结构：计算模型的物理实现

■ 冯·诺依曼体系结构

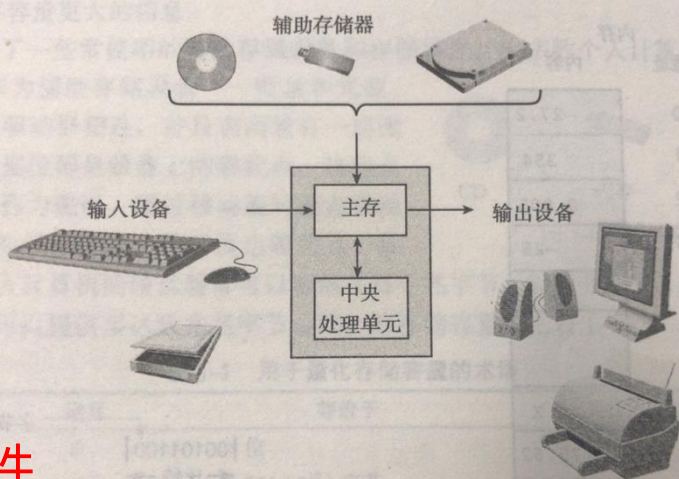
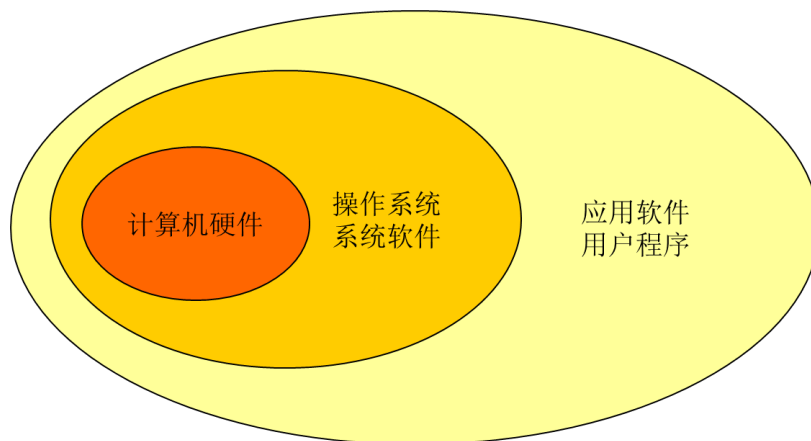
- 计算机硬件由运算器、控制器、存储器、输入/输出设备组成
- 数据和程序以二进制代码形式存放
- 控制器根据存放在存储器中的程序来工作



冯·诺依曼体系结构计算机是一种通用设计，只需修改存储器中的指令序列，就能控制计算机做不同的事情

8.1.1 基础1——计算机与程序运行

■ 计算机系统的构成



硬件

■ 系统软件：计算机工作时必须配备的软件

- 操作系统：管理计算机硬件资源，控制程序运行，为用户提供交互界面
- 语言处理程序：将高级语言源程序翻译成计算机能识别的目标程序
- 数据库管理系统：存储和管理海量数据，方便进行查询和增删改操作

■ 应用软件：为某种特定用途而开发的软件

- 文本编辑：UltraEdit, Notepad++, Word,
- 图像处理：Photoshop, ACDSee, 美图秀秀,
- 视频播放：暴风影音, 优酷视频, 腾讯视频,
- 网页浏览：Chrome, Edge, Firefox, Safari,
-

软件

8.1.1 基础1——计算机与程序运行

■ 信息在计算机中的表示

■ 信息与数据

- 信息：各种事物的变化和特征的反映
- 数据：信息的载体，如数值、文字、图像等
- 人理解的是信息，计算机处理的是数据

■ 计算机中的数据

- 数值数据：表示量的大小、正负，如整数、小数等
- 字符数据：表示一些符号、标记，如英文字母A~Z、a~z、数字0~9、各种专用字符+、-、/、（）.....及标点符号等
- 其他数据：声音数据、图像数据等（需要特定的编码/解码规则）

■ 数据单位

1 B = 8 bit

1 KB = 1024 B = 2^{10} B

1 MB = 1024 KB = 2^{20} B

1 GB = 1024 MB = 2^{30} B

1 TB = 1024 GB = 2^{40} B

■ 计算机科学常用的进制

- 二进制 (Binary)
- 八进制 (Octonary)
- 十六进制 (Hexadecimal)

8.1.1 基础1——计算机与程序运行

■ 进制转换

十进制数 → 非十进制数

例： $(81.65)_{10} = (?)_2$ ，要求精度为小数六位。

整数与小数分别转换

$(0.65)_{10} = (?)_2$ ，**进位法**—乘基数取整。

得： $(0.65)_{10} = (0.101001)_2$

$(81)_{10} = (?)_2$ ，**余数法**—除基取余法

得： $(81)_{10} = (1010001)_2$

综合得： $(81.65)_{10} = (1010001.101001)_2$

注意：小数转换不一定能算尽，只能算到一定精度的位数为止，故要产生一些误差。当位数较多时，这个误差就很小了。

非十进制数 → 十进制数

利用多项式表示法（**位权多项式法**）把各非十进制数按权展开求和。

转换公式：

$$(S)_N = \sum_n^{m} k_i N^{i-1}$$

$$(S)_N = (k_n k_{n-1} \cdots k_1 \cdot k_0 k_{-1} \cdots k_{-m})_N$$

例： $(1011.1)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1}$

$$= 8 + 0 + 2 + 1 + 0.5$$
$$= (11.5)_{10}$$

8.1.1 基础1——计算机与程序运行

■ 进制转换

二进制与八进制之间的转换

从**小数点**开始，将二进制数的整数和小数部分**每三位**分为**一组**，**不足**三位的分别在整数的最高位前和小数的最低位后加“0”补足，然后每组用等值的八进制码替代，即得目的数。

例：(11010111.0100111)_B = (?)

0 11010111.0100111 00

3 2 7 2 3 4

三位合一位

得：(11010111.0100111)_B = (327.234)_O

(327.234)_O = (?)_B

小数点为界

一位拆三位

二进制与十六进制之间的转换

从**小数点**开始，将二进制数的整数和小数部分**每四位**分为**一组**，**不足**四位的分别在整数的最高位前和小数的最低位后加“0”补足，然后每组用等值的十六进制码替代，即得目的数。

例9：111011.10101 B = 3B.A8 H

00111011.10101000

3 B A 8

四位合一位

一位拆四位

8.1.1 基础1——计算机与程序运行

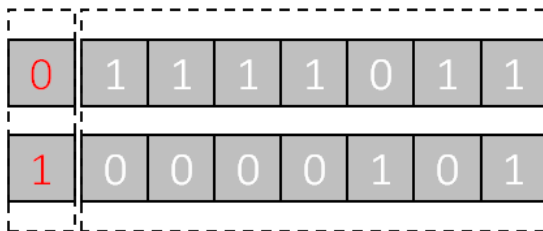
■ 数值的表示

■ 真值与机器数

- 真值：在计算机外部，用正负号表示的实际数值
- 机器数：在计算机内部，将正负号数字化后得到的数值编码结果

$(+123)_D$

$(-123)_D$



符号位

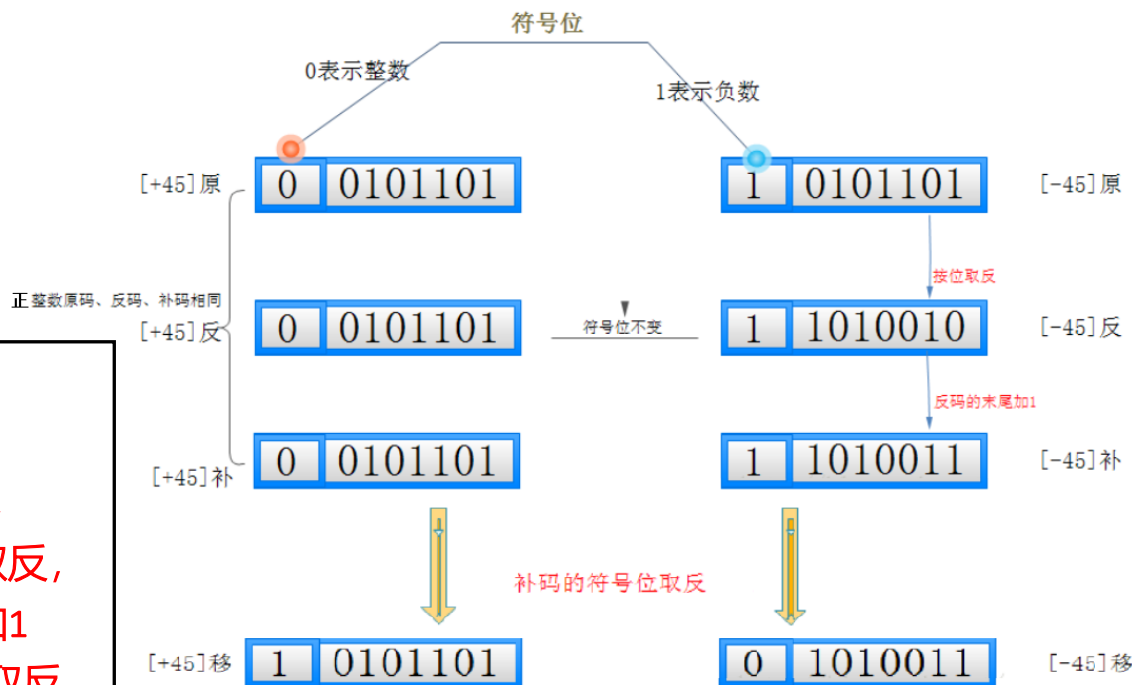
■ 定点数与浮点数

- 定点数：约定所有数值数据的小数点隐含在某一个固定位置上
- 浮点数：小数点的位置可按需要浮动

定点数的表示

规律：

- 正整数的原码、反码、补码相同
- 负整数的原码符号位为1，其余位不变，
- 负整数的反码在其原码基础上，按位取反，
- 负整数的补码在其反码基础上，末位加1
- 正/负整数移码在补码基础上，符号位取反



定点整数的扩展与截断

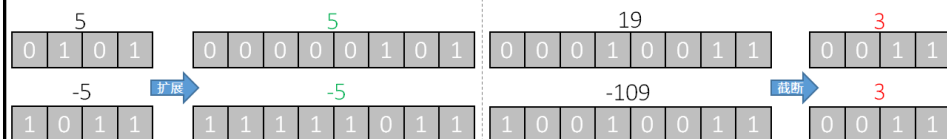
假设一个二进制整数有 ω 位，位向量 $\vec{x} = [x_{\omega-1}, x_{\omega-2}, \dots, x_0]$ ，现欲将其变成 ω' 位，位向量 $\vec{x}'$

■ 扩展 (当 $\omega' > \omega$ 时)：从数值角度考虑 (保值)

- 无符号整数的零扩展： $\vec{x}' = [0, \dots, 0, x_{\omega-1}, x_{\omega-2}, \dots, x_0]$
- 有符号整数补码表示形式的符号扩展： $\vec{x}' = [x_{\omega-1}, \dots, x_{\omega-1}, x_{\omega-1}, \dots, x_0]$

■ 截断 (当 $\omega' < \omega$ 时)：从位级角度考虑 (不保值)

- 截断无符号整数和有符号整数： $\vec{x}' = [x_{\omega'-1}, x_{\omega'-2}, \dots, x_0]$
- 规律： $x' = x \bmod 2^{\omega'}$



有符号整数和无符号整数之间的转换

■ 从数值角度考虑

- 对于在两种形式中都能表示的值，保持不变
- 将负数转换成无符号数，可能会得到0
- 待转换的无符号数超出补码能表示的范围，可能会得到所能表示的最大值

■ 从位级角度考虑 (大多数C语言的实现)

- 规则：保持位值不变，只是改变了解释这些位的方式，从而数值可能改变
- 无符号整数转换为有符号整数的补码表示形式

$$U2T_{\omega}(x) = \begin{cases} x, & x \leq 2^{\omega-1} - 1 \\ x - 2^{\omega}, & x > 2^{\omega-1} - 1 \end{cases} \quad (0 \leq x \leq 2^{\omega} - 1)$$

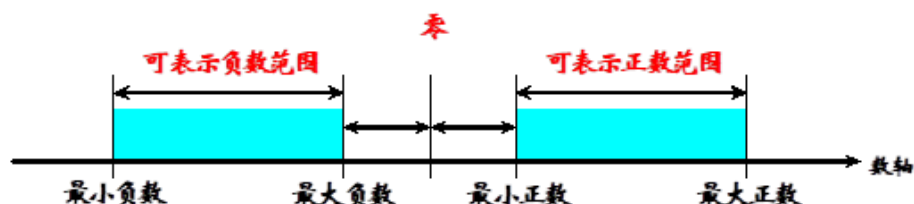
■ 有符号整数的补码表示形式转换为无符号整数

$$T2U_{\omega}(x) = \begin{cases} x + 2^{\omega}, & x < 0 \\ x, & x \geq 0 \end{cases} \quad (-2^{\omega} \leq x \leq 2^{\omega} - 1)$$

无符号整数：19
0 0 0 1 0 0 1 1
有符号整数的补码形式：19

无符号整数：147
1 0 0 1 0 0 1 1
有符号整数的补码形式：-109

浮点数的表示



■ 科学计数法： $N = M \times R^E$

■ M ：尾数， R ：基值， E ：阶码

■ 示例： $0.00303 = 3.03 \times 10^{-3} = 0.303 \times 10^{-2}$

$19880000 = 1.988 \times 10^7 = 0.1988 \times 10^8$

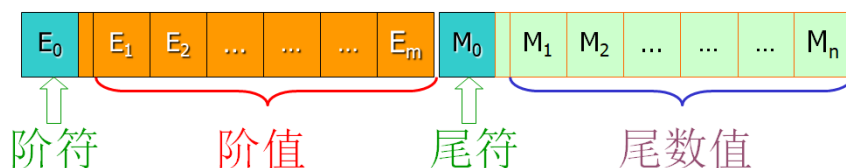
■ 一个机器浮点数由尾数和阶码及其符号位组成

■ 尾数：用定点小数表示，给出有效数字的位数，决定了浮点数的表示精度

■ 阶码：用定点整数表示，指明小数点所在位置，决定了浮点数的表示范围

■ 尾数越长，表示的精度越高；阶码越长，表示的范围越广

■ 在固定长度的浮点数格式内，表示范围和表示精度是一对矛盾



浮点数的规格化

■ 目的：为了数据表示的唯一性，为了提高数据的表示精度

■ 规格化要求：尾数的最高有效位为1，从而保证有n个有效数字

■ 规格化通过尾数移位和修改阶码实现

■ 尾数的小数点每向左移动1位，阶码相应加1

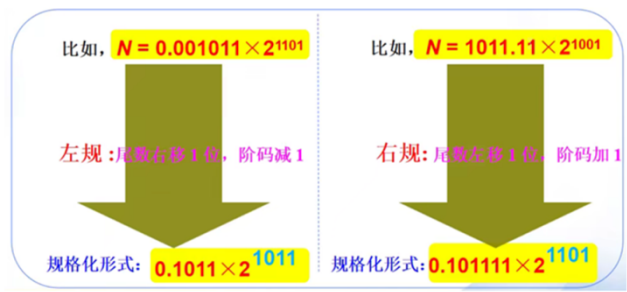
■ 尾数的小数点每向右移动1位，阶码相应减1

• 二进制原码的规格化数：

- 正数：0.1xxxxxxx
- 负数：1.1xxxxxxx

• 二进制补码的规格化数：

- 正数：0.1xxxxxxx
- 负数：1.0xxxxxxx



最大正数： $\text{Max}(+) = S_{\max} \times r^{j_{\max}}$
 $= 0.11...1 \times 2^{11...1}$
 $= (1-2^{-n}) \times 2^{2^m-1}$

最小负数： $\text{Min}(-) = -S_{\max} \times r^{j_{\max}}$
 $= -0.11...1 \times 2^{11...1}$
 $= -(1-2^{-n}) \times 2^{2^m-1}$

最小正数： $\text{Min}(+) = S_{\min} \times r^{j_{\min}}$
 $= 0.00...01 \times 2^{-11...1}$
 $= (2^{-n}) \times 2^{-(2^m-1)}$

最大负数： $\text{Max}(-) = -S_{\min} \times r^{j_{\max}}$
 $= -0.00...01 \times 2^{-11...1}$
 $= -(2^{-n}) \times 2^{-(2^m-1)}$

非规格化形式

最大正数： $\text{Max}(+) = S_{\max} \times r^{j_{\max}}$
 $= 0.11...1 \times 2^{11...1}$
 $= (1-2^{-n}) \times 2^{2^m-1}$

最小负数： $\text{Min}(-) = -S_{\max} \times r^{j_{\max}}$
 $= -0.11...1 \times 2^{11...1}$
 $= -(1-2^{-n}) \times 2^{2^m-1}$

最小正数： $\text{Min}(+) = S_{\min} \times r^{j_{\min}}$
 $= 0.10...0 \times 2^{-11...1}$
 $= (2^{-1}) \times 2^{-(2^m-1)}$

最大负数： $\text{Max}(-) = -S_{\min} \times r^{j_{\max}}$
 $= -0.10...0 \times 2^{-11...1}$
 $= -(2^{-1}) \times 2^{-(2^m-1)}$

规格化形式

8.1.1 基础1——计算机与程序运行

■ 字符的表示

ASCII 值	控制字符	二进制	十进制	十六进制	图形	Binary	Decimal	Hex	Graphic	Binary	Decimal	Hex	Graphic
0	NUL	0010 0000	32	20	(空格) (↵)	0100 0000	64	40	␣	0110 0000	96	60	ˆ
1	SOH	0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97	61	a
2	STX	0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98	62	b
3	ETX	0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99	63	c
4	EOT	0010 0100	36	24	\$	0100 0100	68	44	D	0110 0100	100	64	d
5	ENQ	0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101	65	e
6	ACK	0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102	66	f
7	BEL	0010 0111	39	27	'	0100 0111	71	47	G	0110 0111	103	67	g
8	BS	0010 1000	40	28	(	0100 1000	72	48	H	0110 1000	104	68	h
9	HT	0010 1001	41	29	)	0100 1001	73	49	I	0110 1001	105	69	i
10	LF	0010 1010	42	2A	*	0100 1010	74	4A	J	0110 1010	106	6A	j
11	VT	0010 1011	43	2B	+	0100 1011	75	4B	K	0110 1011	107	6B	k
12	FF	0010 1100	44	2C	,	0100 1100	76	4C	L	0110 1100	108	6C	l
13	CR	0010 1101	45	2D	-	0100 1101	77	4D	M	0110 1101	109	6D	m
14	SO	0010 1110	46	2E	.	0100 1110	78	4E	N	0110 1110	110	6E	n
15	SI	0010 1111	47	2F	[	0100 1111	79	4F	O	0110 1111	111	6F	o
16	DLE	0011 0000	48	30	0	0101 0000	80	50	P	0110 0000	112	70	p
17	DC1	0011 0001	49	31	1	0101 0001	81	51	Q	0110 0001	113	71	q
18	DC2	0011 0010	50	32	2	0101 0010	82	52	R	0110 0010	114	72	r
19	DC3	0011 0011	51	33	3	0101 0011	83	53	S	0110 0011	115	73	s
20	DC4	0011 0100	52	34	4	0101 0100	84	54	T	0110 0100	116	74	t
21	NAK	0011 0101	53	35	5	0101 0101	85	55	U	0110 0101	117	75	u
22	SYN	0011 0110	54	36	6	0101 0110	86	56	V	0110 0110	118	76	v
23	TB	0011 0111	55	37	7	0101 0111	87	57	W	0110 0111	119	77	w
24	CAN	0011 1000	56	38	8	0101 1000	88	58	X	0110 1000	120	78	x
25	EM	0011 1001	57	39	9	0101 1001	89	59	Y	0110 1001	121	79	y
26	SUB	0011 1010	58	3A	:	0101 1010	90	5A	Z	0110 1010	122	7A	z
27	ESC	0011 1011	59	3B	;	0101 1011	91	5B	[	0110 1011	123	7B	{
28	FS	0011 1100	60	3C	<	0101 1100	92	5C	\	0110 1100	124	7C	
29	GS	0011 1101	61	3D	=	0101 1101	93	5D	]	0110 1101	125	7D	}
30	RS	0011 1110	62	3E	>	0101 1110	94	5E	^	0110 1110	126	7E	~
31	US	0011 1111	63	3F	?	0101 1111	95	5F	_				
127	DEL												

图：国标码(4D3C)_H (0100110100111100)_B
机内码(CDBC)_H (1100110110111100)_B

灵：国标码(4169)_H (0100000101101001)_B
机内码(C1E9)_H (1100000111101001)_B

■ 西文字符：ASCII编码

- **ASCII** (American Standard Code for Information Interchange,美国信息交换标准码)。通用的是7位ASCII码，用7位二进制数表示一个字符的编码。共有 $2^7=128$ 个不同的编码
- 主要的字符及其编码：
 - 48 ~ 57为10个阿拉伯数字0 ~ 9
 - 65 ~ 90为26个大写英文字母A ~ Z
 - 97 ~ 122为26个小写英文字母a ~ z
 - 32为空格，13为回车

■ 中文字符：

- **国标码**：常用汉字6763个，其中一级汉字3755个，二级汉字3008个。国标码用两个字节来表示一个汉字，每个字节的最高位为0
- **机内码**：在计算机内部对汉字进行存储、处理的编码。一个汉字的内码用两个字节存储，每个字节的最高位为1，以区别于ASCII码



8.1.1 基础1——计算机与程序运行

■ C语言介绍

■ C语言是一门面向过程的编译型“高级语言”。它既有高级语言的特点，又具有低级汇编语言的特点

■ C语言可以作为系统设计语言，编写工作系统应用程序，也可以作为应用程序设计语言，编写不依赖计算机硬件的应用程序

■ 为什么学习C语言？

■ C语言相比其它高级语言如（C++，Java，C#）是低级语言，通过它可以更好地了解计算机底层工作原理。比如数据在内存中是如何存储的，如何直接访问内存中的数据等等。

■ C语言是其他任何高级语言的基础，学好C语言，就可以更容易掌握其他语言。语言都是相通的，C更专注于语言的实质，而不需要分散更多精力在集成开发环境的使用和抽象的数据概念上。

■ 功能强大、适用范围大、可移植性好

- 许多著名的系统软件都是由C语言编写的。C语言可以像汇编语言一样对位、字节和地址进行操作，而这三者是计算机最基本的工作单元。
- C语言适合于多种操作系统，如DOS、UNIX等。对于操作系统、系统使用程序以及需要对硬件进行操作的场合，用C语言明显优于其它解释型高级语言，一些大型应用软件也是用C语言编写的。

■ 运算符丰富

- C语言的运算符包含的范围很广泛，共有34种运算符。C语言把括号、赋值、强制类型转换等都作为运算符处理。
- C语言的运算类型极其丰富，表达式类型多样化。灵活使用各种运算符可以实现在其它高级语言中难以实现的运算。

■ 数据结构丰富

- C语言的数据类型有：整形、实型、字符型、数组类型、指针类型、结构体类型、共用体类型等。能用来实现各种复杂的数据结构的运算。
- 引入了指针概念，使程序效率更高，甚至可以直接访问内存地址。

```
1. #include <stdio.h> /*包含标准输入输出头文件*/
2. #define C 1 /*定义常量C的值为1*/
3. int main(int argc, char **argv) /*主函数*/
4. { /*函数体开始*/
5.     int x; /*定义整数类型变量x*/
6.     scanf("%d", &x); /*调用库函数，读入一个整数值，存入变量x中*/
7.     x = x + C; /*计算x+C的值，结果存入变量x中*/
8.     printf("Hello World: %d", x) /*调用库函数，输出结果*/
9.     return 0; /*返回值*/
10. }
```

8.1.2 语法1——程序设计基本概念

■ 简单C程序的基本结构

计算 $1!+2!+\dots+n!$ 的C语言代码 (first.c)

```
1. #include <stdio.h>           //引用头文件
2. #define SUM_INIT 0           //定义符号常量SUM_INIT

3. int z = SUM_INIT;            /* 定义全局变量z, 并进行初始化 */

4. int main(int argc, char **argv) {
5.     int x = 1;                /* 定义局部变量x, 并进行初始化 */
6.     int i, n;                 /* 定义局部变量i和n, 未进行初始化 */

7.     scanf("%d", &n);          //输入
8.     for (i = 1; i <= n; i++) { //L8-11行: 循环语句
9.         x = x * i;
10.        z = z + x;
11.    }

12.    printf("%d\n", z);         //输出
13.    return 0;                 //函数返回值
14.}
```

第一个C程序
(first.c)

#include
#define

预处理命令

全局声明

int main()

主函数：总是第一个被调用

statements

语句：实现主函数功能

expressions

表达式：实现值的计算

注释：//开始的单行与/* */的多行

8.1.2 语法1——程序设计基本概念

■ 数据类型

■ 计算机中的各种数据是存储在内存空间中

- 整数、实数、字符.....
- 不同类型的数据占用大小不同的内存空间

■ 数据类型可分为两大类

- 基本数据类型：整数型，浮点型，字符型
- 构造数据类型：数组、枚举、结构体、共用体
- 注：构造数据类型，是由若干个基本数据类型的变量按特定的规律组合构造而成的

基本数据类型：整数

- 有符号(signed)与无符号(unsigned)的区分
- 表示整数使用int, short, long, long long四种类型
- 表示实数使用float, double, long double三种类型
- 表示字符使用char类型

■ 多字节数据的存储与排列顺序

- 示例：int i = -65535，存放在100号单元（占100 ~ 103），则用“取数”指令访问100号单元取出i时，必须清楚i的4个字节是如何存放的

$$65535 = 2^{16} - 1$$

$$[-65535]_{\text{补}} = \text{FFFF0001H}$$



类型	占用内存字节数 (sizeof)	值域	
		signed	unsigned
char	1	$[-2^7, 2^7-1]$ $([-128, 127])$	$[0, 2^8-1]$ $([0, 255])$
short	2	$[-2^{15}, 2^{15}-1]$ $([-32768, 32767])$	$[0, 2^{16}-1]$ $([0, 65535])$
int	4	$[-2^{31}, 2^{31}-1]$ $([-2147483648, 2147483647])$	$[0, 2^{32}-1]$ $([0, 4294967295])$
long	8	$[-2^{63}, 2^{63}-1]$	$[0, 2^{64}-1]$
long long	8	$[-2^{63}, 2^{63}-1]$	$[0, 2^{64}-1]$
float	4 (有效数字: 6)	$\{0, [-3.4 \times 10^{38}, -1.2 \times 10^{-38}], [1.2 \times 10^{-38}, 3.4 \times 10^{38}]\}$	
double	8 (有效数字: 15)	$\{0, [-1.7 \times 10^{308}, -2.3 \times 10^{-308}], [2.3 \times 10^{-308}, 1.7 \times 10^{308}]\}$	
long double	16 (有效数字: 19)	$\{0, [-1.1 \times 10^{4932}, -3.4 \times 10^{-4932}], [3.4 \times 10^{-4932}, 1.1 \times 10^{4932}]\}$	

注：以上是在64位Ubuntu系统上的结果，不同平台可能略有差异。可以用sizeof操作符查看占用内存字节数。

基本数据类型：实数

■ 例1：若浮点数 x 的二进制存储格式为 $(41360000)_H$ ，求其32位浮点数的十进制值

解：0100,0001,0011,0110,0000,0000,0000,0000

数符：0

阶码：1000,0010

尾数：011,0110,0000,0000,0000,0000

指数 $e = \text{阶码} - 127 = 10000010 - 01111111 = 00000011 = (3)_D$

包括隐藏位1的尾数： $1.M = 1.011\ 0110\ 0000\ 0000\ 0000\ 0000 = 1.011011$

于是有 $x = (-1)^s \times 1.M \times 2^e$

$= + (1.011011) \times 2^3 = (1011.011)_B = (11.375)_D$

■ 例2：将十进制数20.59375转换成32位浮点数的二进制格式存储

解：首先分别将整数和分数部分转换成二进制数：

$(20.59375)_D = (10100.10011)_B$

然后移动小数点，使其在第1，2位之间：

$10100.10011 = 1.010010011 \times 2^4$

于是得到：

$S = 0, E = e + 127 = 4 + 127 = 131 = 1000,0011, M = 010010011$

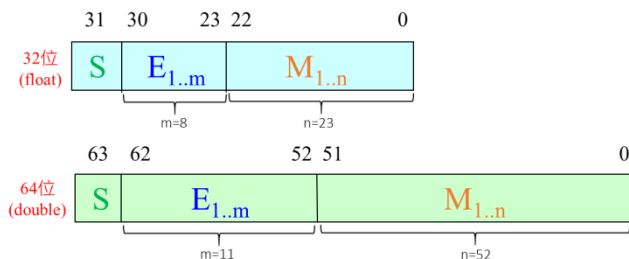
最后得到32位浮点数的二进制存储格式为：

0100 0001 1010 0100 1100 0000 0000 0000 = $(41A4C000)_{16}$

■ 浮点数的标准格式IEEE754

- 尾数 (M)：用原码表示，规格化后使用隐藏位技术，尾数符号在数据首位
- 阶码 (E)：用移码表示，偏移值为 $2^{m-1} - 1$ (float偏移量127, double偏移量1023)
- 基值 (R)：等于2

科学计数法： $N = M \times R^E$



■ float类型的特殊值

- 正零：0 00000000 00000000 00000000 00000000
- 负零：1 00000000 00000000 00000000 00000000
- 正无穷：0 11111111 00000000 00000000 00000000
- 负无穷：1 11111111 00000000 00000000 00000000
- 不合法数：* 11111111 (23位小数部分不全为0)

■ float类型的范围

- 负最小值：1 11111110 11111111 11111111 11111111 = $-(1.\text{ffffe})_H \times 2^{127} \approx -3.4 \times 10^{38}$
- 负最大值：1 00000001 00000000 00000000 00000000 = $-(1.0)_H \times 2^{-126} \approx -1.2 \times 10^{-38}$
- 正最小值：0 00000001 00000000 00000000 00000000 = $+(1.0)_H \times 2^{-126} \approx 1.2 \times 10^{-38}$
- 正最大值：0 11111110 11111111 11111111 11111111 = $+(1.\text{ffffe})_H \times 2^{127} \approx 3.4 \times 10^{38}$
- 提示：阶码全0和全1保留做特殊值，尾数还有一个隐藏位

最大正数：Max(+) = $S_{\max} \times r^{j_{\max}}$ $= 0.11...1 \times 2^{11...1}$ $= (1-2^{-n}) \times 2^{2^m-1}$	最小正数：Min(+) = $S_{\min} \times r^{j_{\min}}$ $= 0.10...0 \times 2^{-11...1}$ $= (2^{-1}) \times 2^{-(2^m-1)}$
最小负数：Min(-) = $-S_{\max} \times r^{j_{\max}}$ $= -0.11...1 \times 2^{11...1}$ $= -(1-2^{-n}) \times 2^{2^m-1}$	最大负数：Max(-) = $-S_{\min} \times r^{j_{\min}}$ $= -0.10...0 \times 2^{-11...1}$ $= -(2^{-1}) \times 2^{-(2^m-1)}$

规格化形式

■ float类型的有效数字（数值精确的数位）

- float类型有23个二进制位用于存放尾数，另有一个隐含位固定为1（无灵活度）
- $2^{23} = 8388608 \approx 0.84 \times 10^7$
- float类型最多能有7个十进制有效数字，但能绝对保证的是6个有效数字

基本数据类型：字符

- 占用1个字节字符类型的数据，在内存中以相应的ASCII码存放
- 例：'1'的ASCII码为(49)_D，内存中存储的是ASCII码值

0	0	1	1	0	0	0	1
---	---	---	---	---	---	---	---

字符'1'在内存中的存储

0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

整数1在内存中的存储

- 注意：
 - 字符'1'与整数1是不同的概念
 - 字符'1'代表一个形状为'1'的符号，在内存中以ASCII码形式存储，占1个字节
 - 整数1是一个数值，在内存中以二进制补码形式存储，占2/4/8个字节
 - $1 + 1 = 2$, $'1' + '1' \neq 2$
 - char类型除了表示字符，还可以看作是一个特殊的整数类型
 - 1个字节的整数
 - char的范围[-128, 127]，unsigned char的范围[0, 255]

8.1.2 语法1——程序设计基本概念

■ 整型字面常量

- 默认：一个整型字面常量，默认是十进制表示的 int 类型（若范围允许）
- 前缀：改变当前字面常量的进制（0x/0X: 十六进制，0: 八进制）
- 后缀：改变当前字面常量的类型（U/u: unsigned, L/l: long, LL/ll: long long）

```
1. #include <stdio.h>
2. int main(int argc, char **argv) { /* sizeof(x): 获得x所占用的内存字节数 */
3.     printf("%d %d %d\n", 12, 012, 0x12);
4.     printf("%d %d %d %d\n", sizeof(0x12), sizeof(0x12U), sizeof(0x12L), sizeof(0x12ULL));
5.     printf("%d %d %d\n", sizeof(2147483647), sizeof(2147483648), sizeof(2147483648U));
6.     printf("%d %d\n", sizeof(0x80000000), sizeof(0x80000000U));
7. }
```

思考：上面代码的输出是什么？

答案（在64位Ubuntu环境下）：

12 10 18
4 4 8 8
4 8 4
4 4

■ 实型字面常量

- 十进制小数形式：123.456, 0.345, -56.79, 0.0, 12.0,
- 指数形式：12.34e3(12.34×10^3), -346.87e-25(-346.87×10^{-25}),
- 默认：一个实型字面常量，默认是double 类型（若范围允许）
- 后缀：改变当前字面常量的类型（F/f: float, L/l: long double）
- C99标准新增的表示方式：以0x开头，然后是16进制尾数部分，接着是p，后面是以2为底的阶码。例： $0xa.1fp10 = (10 + \frac{1}{16} + \frac{15}{256}) \times 2^{10} = 10364$

```
1. #include <stdio.h>
2. int main(int argc, char **argv) {
3.     float a = 10364, b = 10.364e3, c = 0xa.1fp10;
4.     printf("%f %f %f\n", a, b, c);
5.     printf("%x %x %x\n", *(unsigned int *)(&a), *(unsigned int *)(&b), *(unsigned int *)(&c));
6.     printf("%d %d %d\n", sizeof(float), sizeof(double), sizeof(long double));
7.     printf("%d %d %d\n", sizeof(12.34), sizeof(12.34F), sizeof(12.34L));
8. }
```

答案（在64位Ubuntu环境下）：
10364.000000 10364.000000 10364.000000
4621f000 4621f000 4621f000
4 8 16
8 4 16

思考：上面代码的输出是什么？

■ 字符字面常量

- 普通字符：用单引号括起来的字符，如：'a', 'Z', '3', '?',
- 转义字符：用"\ "开头的字符序列表示特殊的字符
- 常用的特殊字符：

- 引号：\'，\"
- 制表符：\t，\v
- 回车换行：\n，\r
- 问号：\?
- 反斜杆：\\

■ 使用转义字符表示

- \o或者\oo或者\ooo：与该八进制码对应的字符（最多3位八进制数）
- \xh或者\xhh：与该十六进制码对应的字符（最多2位十六进制数）

- 例：字符'1'的多种表示方式——'1', '\061', '\x31'

思考：字符字面常量占用的内存字节数是多少？sizeof('a')=?

常量

■ 字符串字面常量

- 用双引号括起来的字符序列，如："hello"
- 字符串常量是双引号中的全部内容，但不包括双引号本身
- 字符串中字符个数称为该字符串的长度
- 字符串常量存储在机器中时，系统自动在其末尾加一个结束标志'\0'

思考：字符串字面常量占用的内存字节数是多少？sizeof("hello")=? strlen("hello")=?

☞ 字符串"hello"存储为：

h	e	l	l	o	\0
---	---	---	---	---	----

☞ 实际上每个字符都以ASCII码存储：

104	101	108	108	111	0
-----	-----	-----	-----	-----	---

8.1.2 语法1——程序设计基本概念

■ 变量

■ 变量命名规则

- 变量名可包含字母、数字和下划线，但只能以字母或下划线开头
- C语言的关键字不能作为变量名
- 变量名是大小写敏感的
- 变量在一个函数范围内不能重名

■ 变量赋值的关键要素

- 变量必须先定义再使用
- 在变量定义时可对其赋初值，这叫变量初始化，是一种良好的编程习惯
- 对变量的赋值过程是“覆盖”写入，在变量地址单元中用新值去替换旧值
- 读取变量的值，该变量保持不变，相当于拷贝一份出来
- 参与表达式运算的所有变量都保持原来的值不变

8.1.2 语法1——程序设计基本概念

■ 运算符和表达式

■ 运算符涉及的操作数个数

- 单目运算符（一元运算符）：只有1个操作数。
- 双目运算符（二元运算符）：具有2个操作数。（主要类型）
- 三目运算符（三元运算符）：需要3个操作数。（唯一：条件运算符 `?:`）

■ 运算符的优先级

- 某些运算符先于其他运算符被执行
 - 例如， $x + y * 4$ ，先乘除后加减。
- 必要时可用圆括号 `()` 改变计算顺序
 - 例如，求三个数的平均值。
 - 错误的写法： $a + b + c / 3$
 - 正确的写法： $(a + b + c) / 3$

■ 运算符的结合性：

- 出现并列的优先级别相同的运算符时，由运算符的结合性决定计算的次序
- 常见的算术运算符左结合，赋值运算符右结合
 - 例： $x * y / z$ 相当于 $(x * y) / z$
 - 例： $x = y = z + 1$ 相当于 $x = (y = z + 1)$

8.1.2 语法1——程序设计基本概念

算术运算符:	+ - * / % ++ --
赋值运算符:	= 及其扩展
求字节数 :	sizeof
强制类型转换:	(类型)
逗号运算符:	,
函数调用符运算符:	()
关系运算符:	< <= == > >= !=
逻辑运算符:	! &&
条件运算符:	?:
下标运算符:	[]
分量运算符:	. ->
位运算符 :	<< >> ~ & ^
指针运算符:	* &

算术运算符

```
1. #include <stdio.h>
2. int main() {
3.     int i=6, a, b;

4.     printf("%d ", ++i);
5.     printf("%d ", i++);

6.     a=--i; printf("%d ", a);
7.     b=i--; printf("%d\n", b);

8.     printf("%d ", -i++);
9.     printf("%d ", ++i);
10.    printf("%d\n", i);

11.    i = 8;
12.    printf("%d ", ++i + i++);
13.    printf("%d\n", i);

14.    i = 8;
15.    printf("%d %d\n", i, i++ + i++);
16.}
```

■ 自增和自减运算符

- 自增运算符：++，将操作数的值增1
- 自减运算符：--，将操作数的值减1
- 操作数必须是整型和字符型变量
- 单目运算符
- 优先级高于常见的算术运算符
- 结合性：右结合
- 使用时的注意事项：
 - ++ (--) 在前：先加（减）后用
 - ++ (--) 在后：先用后加（减）

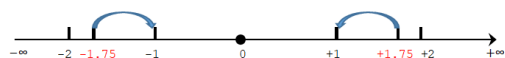
运算符名称	算术运算符	代数表达式	C语言表达式	适用的数据类型
正号	+	+ a	+ a	整数、字符、浮点数
负号	-	- b	- b	整数、字符、浮点数
加	+	f + 7	f + 7	整数、字符、浮点数
减	-	p - c	p - c	整数、字符、浮点数
乘	*	b m	b * m	整数、字符、浮点数
除	/	x / y	x / y	整数、字符、浮点数
取模	%	r mod s	r % s	整数、字符

■ 正负号是单目运算符，其余是双目运算符

■ 优先级：

- 第1优先级：正号、负号
- 第2优先级：乘、除、取模
- 第3优先级：加、减

例：7 ÷ 4 = 1.75 (-7) ÷ 4 = -1.75 7 ÷ (-4) = -1.75 (-7) ÷ (-4) = 1.75



7/4=1 (-7)/4=-1 7/(-4)=-1 (-7)/(-4)=1
7%4=3 (-7)%4=-3 7%(-4)=3 (-7)%(-4)=-3

■ 结合性：左结合

■ 除号运算符的使用问题：

- 两个操作数全为整型数（包括char、short、int、long、long long）时，为整除运算
- 当有任一操作数为实型数时，为普通除法运算
- C99标准规定整数除法采用“**趋零截尾**”，向0方向取最接近精确值的整数，换言之就是舍去小数部分

■ 取模运算符的使用问题：

- 操作数只能整型数据（包括char、short、int、long、long long）
- C语言中的取模运算：x % y = x - x/y*y

答案（GCC编译器）：

7□7□7□7
-6□-8□8
19□10
10□17

答案（Visual Studio 2019编译器）：

7□7□7□7
-6□-8□8
18□10
10□16

注意：C标准规定了自增/自减运算符的增/减操作在一个完整的表达式计算后完成，但没有规定在哪个子表达式计算后进行。
不同的编译器

原则：

- 当一个变量多次出现在一个表达式里时，不要对其进行自增/自减操作
- 若一个变量出现在同一个函数调用的多个参数中，不要对其进行自增/自减操作

赋值运算符

■ 赋值运算符

- 简单赋值运算符：=
- 复合赋值运算符：+=, -=, *=, /=, %=
- 优先级：低于算术运算符
- 结合性：右结合

■ 赋值表达式

<变量> <赋值运算符> <表达式>

d = 23

- 作用：将表达式的值赋给变量
- 左值和右值：
 - 左值是可寻址的变量，可以出现在等号左边或者右边
 - 右值是不可寻址的常量，或在表达式求值过程中创建的临时量，只能出现在等号右边
 - 例：x = y + 1;
 - x做左值，代表的是存储变量x的内存单元（地址）
 - y做右值，代表的事存储变量y的内存单元的值
- 赋值表达式的值就是被赋值的变量的值

■ 简单赋值运算符：=

举例

```
c=a+b  
a=b=c=d=10  
x=(a=5)+(b=8)
```



```
a=(a+b)  
a=(b=(c=(d=10)))  
a=5, b=8, x=a+b
```

■ 复合赋值运算符：+=, -=, *=, /=, %=

■ 简化了赋值表达式

<variable> <operator> = <expression>

- 由下面的表达式简化而来

<variable> = <variable> <operator> <expression>

举例

```
a+=5  
x*=y+7  
x+=x-=x*=x
```



```
a=a+5  
x=x*(y+7)  
x=x+(x=x-(x=x*x))
```

8.1.2 语法1——程序设计基本概念

■ 类型转换

■ 隐式类型转换：处理不同类型的数据时，计算机自动进行转换

- 运算转换：不同类型数据混合运算时
- 赋值转换：把一个值赋给与其类型不同的变量时
- 输出转换：输出时转换成指定的输出格式
- 函数调用转换：实参与形参类型不一致时转换

■ 显式类型转换：程序员在代码中使用强制类型转换运算符

- 强制类型转换运算：（类型标识符）表达式
- 强制类型转换是一个单目运算符，各种数据类型都可用于显式转换运行符

- 例：

```
(char) ( 3 - 3.14159 * x )  
k=(int) ((int)x + (float)i + j)  
(float) (x = 99)  
(double) (5 % 3)
```

■ 注意：

- 表达式应该用括号括起来：(int) x + y只将x转换成整型，然后与y相加
- 对一个变量进行显式转换后，得到一个所需类型的中间变量，原来变量类型不变

8.1.2 语法1——程序设计基本概念

■ 数学函数

- 求绝对值函数
- 指数和对数函数
- 三角函数
- 取整函数
-

注：需引用头文件 `#include <math.h>`，编译时链接相应的库

■ abs函数

- 函数原型: `int abs(int i);`
- 功能: 返回整数的绝对值。

■ fabs函数

- 函数原型: `double fabs(double x);`
- 功能: 返回浮点数的绝对值。

■ ceil函数

- 函数原型: `double ceil(double x);`
- 功能: 向上舍入，返回不小于x的最小整数。

■ floor函数

- 函数原型: `double floor(double x);`
- 功能: 向下舍入，返回不大于x的最大整数。

■ round函数

- 函数原型: `double round(double x);`
- 功能: 四舍五入，返回x的四舍五入整数值。

■ sqrt函数

- 函数原型: `double sqrt(double x);`
- 功能: 计算平方根，返回x的平方根，x应大于等于0

■ exp 函数

- 函数原型: `double exp(double x);`
- 功能: 返回指数函数ex的值。

■ pow 函数

- 函数原型: `double pow(double x, double y);`
- 功能: 返回指数函数(x的y次方)的值。

■ log 函数

- 函数原型: `double log(double x);`
- 功 能: 返回自然对数函数ln(x)（即logex）的值。

■ log10函数

- 函数原型: `double log10(double x);`
- 功 能: 返回以10为底的对数函数（即log10x）的值。

■ 标准库函数：C编译系统为方便用户使用而提供的公共函数

- 输入/输出函数
- 数学函数
- 字符串函数
-

■ 标准库参考手册

C 标准库 - <math.h>

简介

math.h 头文件定义了各种数学函数和一个宏。在这个库中所有可用的功能都带有一个 `double` 类型的参数，且都返回 `double` 类型的结果。

库函数

下面列出了头文件 math.h 中定义的函数：

序号	函数 & 描述
1	<code>double acos(double x)</code> 返回以弧度表示的 x 的反余弦。

■ sin函数

- 函数原型: `double sin(double x);`
- 功能: 正弦函数，返回x的正弦（即sin(x)）的值，x的单位为弧度。

■ asin函数

- 函数原型: `double asin(double x);`
- 功能: 反正弦函数，返回x的反正弦（即sin-1(x)）的值，x应在-1到1范围内。

■ cos函数

- 函数原型: `double cos(double x);`
- 功能: 余弦函数，返回x的余弦（即cos(x)）的值，x的单位为弧度。

■ acos函数

- 函数原型: `double acos(double x);`
- 功能: 反余弦函数，返回x的反余弦（即cos-1(x)）的值，x应在-1到1范围内。

■ tan函数

- 函数原型: `double tan(double x);`
- 功能: 正切函数，返回x的正切（即tan(x)）的值，x为弧度。

■ atan函数

- 函数原型: `double atan(double x);`
- 功能: 反正切函数，返回x的反正切（即tan-1(x)）的值。

8.1.2 语法1——程序设计基本概念

■ 数学函数

■ 练习：编程实现下列计算

1、 计算 $y = 1 + \frac{1}{1 + \frac{1}{1 + \frac{1}{5}}}$

2、 计算 $\sqrt{3^2 + 4^2}$

3、 计算 $\sqrt{\frac{1 - \cos(\pi / 3)}{2}}$

4、 计算 $y = 2 \sin^2(\pi / 4) + \sin(\pi / 4) \square \cos(\pi / 4) - \cos^2(\pi / 4)$

5、 计算 $y = \frac{2\sqrt{5}(\sqrt{6} + \sqrt{3})}{6 + 3}$

6、 计算 $y = \frac{\ln 5(\ln 3) - \ln 2}{\sin(\pi / 3)}$

8.1.2 语法1——程序设计基本概念

■ 语句

■ first.c涉及各种不同类型的语句

```
1. #include <stdio.h>
2. #define SUM_INIT 0

3. int z = SUM_INIT;                //声明定义语句

4. int main(int argc, char **argv) {
5.     int x = 1;                    //声明定义语句
6.     int i, n;                     //声明定义语句

7.     scanf("%d", &n);              //函数调用语句
8.     for (i = 1; i <= n; i++)       //控制语句
    {
9.         x = x * i;                //表达式语句
10.        z = z + x;                 //表达式语句
11.    }                               } //复合语句
12.    printf("%d\n", z);             //函数调用语句
13.    return 0;                      //控制语句
14.}
```

8.1.2 语法1——程序设计基本概念

表达式语句

- 表达式语句由一个表达式加一个分号构成，任何表达式都可以加上分号而成为语句
- 最典型的表达式语句是由赋值表达式构成的赋值表达式语句
 - 赋值表达式：a = 3
 - 赋值表达式语句：a = 3;
- 副作用：对程序变量或状态的修改
 - 无副作用的表达式语句：i; x + y;
 - 有副作用的表达式语句：i++; z = x + y;

声明/定义定义语句

- 声明/定义语句用于声明/定义函数或变量，可以全局的或局部的
- 函数声明语句：描述一个函数接口，包括函数名字、形参个数及类型、返回值类型
 - 通知编译器，以便在调用该函数时进行检查
 - 告诉程序设计人员，如何调用该函数

```
int scanf(const char *format, ...);  
int printf ( const char * format, ... );
```
- 变量定义语句：用于为变量分配存储空间，还可为变量指定初始值
 - int x = 1;
 - int i, n;

语句

控制语句

- 选择语句：
 - if () ... else ...: 两个分支
 - switch (): 多个分支
- 循环语句：
 - for () ...
 - while () ...
 - do ... while ()
- 改变执行流程语句：
 - 结束本次循环语句：continue
 - 终止执行switch或循环语句：break
 - 转向语句：goto (慎用)
 - 从函数返回语句：return

复合语句

- 用{}把一些语句和声明括起来成为复合语句（又称为语句块）
- 可以在复合语句中定义局部变量
- 例：

8.1.2 语法1——程序设计基本概念

■ 数据的输入输出

% 附加字符 格式字符

- (1) printf函数输出时，务必注意输出对象的类型应与上述格式说明匹配，否则将会出现错误。
- (2) 除了X,E,G外，其他格式字符必须用小写字母，如%d不能写成%D。
- (3) 可以在printf函数中的格式控制字符串内包含转义字符，如\n,\t,\b,\r,\f和\377等。
- (4) 一个格式声明以“%”开头，以格式字符之一为结束，中间可以插入附加格式字符（也称修饰符）。
- (5) 如果想输出字符“%”，应该在“格式控制字符串”中用连续两个“%”表示，如：
printf("%f%\n",1.0/3);

格式字符	说 明
d,i	以带符号十进制形式输出整数（正数不输出符号）
o	以八进制无符号形式输出整数（不输出前导符0）
x,X	以十六进制无符号形式输出整数（不输出前导符0x），用x则输出十六进制数的a~f时以小写形式输出，用X时，则以大写字母输出
u	以无符号十进制形式输出整数
c	以字符形式输出，只输出一个字符
s	输出字符串
f	以小数形式输出单、双精度数，隐含输出6位小数
e,E	以指数形式输出实数，用e时指数以“e”表示(如1.2e+02)，用E时指数以“E”表示(如1.2E+02)
g,G	选用%f或%e格式中输出宽度较短的一种格式，不输出无意义的0。用G时，若以指数形式输出，则指数以大写表示

附加字符	说 明
l	长整型整数，可加在格式符d、o、x、u前面)
m (代表一个正整数)	数据最小宽度
n (代表一个正整数)	对实数，表示输出n位小数；对字符串，表示截取的字符个数
-	输出的数字或字符在域内向左靠

8.1.2 语法1——程序设计基本概念

■ 数据的输入输出

% 附加字符 格式字符

(1) scanf函数中的格式控制后面应当是**变量地址**，而不是变量名。

应与上述格式说明匹配，否则将会出现错误。

(2)如果在格式控制字符串中除了格式声明以外还有其他字符，则在输入数据时在对应的位置上应输入与这些字符相同的字符。

(3)在用“%c”格式声明输入字符时，空格字符和“转义字符”中的字符都作为有效字符输入。

(4) 在输入数值数据时，如输入空格、回车、Tab键或遇非法字符(不属于数值的字符)，认为该数据结束。

格式字符	说 明
d,i	输入 有符号十进制 整数
u	输入 无符号十进制 整数
o	输入 无符号八进制 整数
x,X	输入 无符号十六进制 整数(大小写作用相同)
c	输入单个字符
s	输入字符串，将字符串送到一个字符数组中，在输入时以非空白字符开始，以 第一个分隔符 (\n、\t、空格) 结束
f	输入实数，可以用小数形式或指数形式输入
e,E,g,G	与f作用相同，e与f、g可以互相替换(大小写作用相同)

附加字符	说 明
l	输入长整型数据（可用%ld，%lo，%lx，%lu）以及 double型数据 (用%lf或%le)
h	输入短整型数据（可用%hd，%ho，%hx）
域宽	指定输入数据所占宽度（列数），域宽应为正整数，注意： 输入浮点数时不能规定精度
*	本输入项在读入后不赋给相应的变量

8.1.2 语法1——程序设计基本概念

■ 数据的输入输出

```
1. #include <stdio.h>
2. int main(int argc, char **argv) {
3.     char c = 'A';
4.     float f = 1234.567890;
5.     char *str = "Hello\0World";
6.     printf("%c %d %o %x\n", c, c, c, c);
7.     printf("%f %e %g\n", f, f, f);
8.     printf("%10d\n%-10d\n%010d\n", c, c, c);
9.     printf("%10.4f\n%10.8f\n%4.6f\n", f, f, f);
10.    printf("%s\n", str);
11. }
```

```
A 65 101 41
1234.567871 1.234568e+003 1234.57
        65
65
0000000065
 1234.5679
1234.56787109
1234.567871
Hello
```

```
1. #include <stdio.h>
2. int main(int argc, char **argv) {
3.     char a, b, c, d, e[100], f;
4.     int g, h, i;
5.     scanf("%c%d%o%x", &a, &b, &c, &d);
6.     printf("%c%c%c%c\n", a, b, c, d);
7.     scanf("%3d%5d%c%d", &g, &h, &i);
8.     printf("%d %d %d\n", g, h, i);
9.     scanf("%s", e);
10.    printf("%s", e);
11. }
```

输入:

A 65 101 41
12345678,90
Hello World

输出:

AAAA
123 45678 90
Hello

8.1.2 语法1——程序设计基本概念

■ 字符/字符串输入输出函数

■ getchar函数：从键盘读入一个字符

- `c=getchar()` 等价于 `scanf("%c", c)`
- `getchar`函数没有参数，函数的值就是从输入设备得到的字符
- `getchar`函数可以从输入设备获得一个可显示字符或者控制字符
- 用`getchar`函数得到的字符可以赋给一个字符变量或整型变量，也可作为表达式的一部分

```
1. #include <stdio.h>
2. int main(int argc, char **argv) {
3.     char a, b;
4.     a = getchar();
5.     b = getchar();
6.     putchar(a);
7.     putchar(b);
8.     putchar(getchar());
9. }
```

输入：BOY

输出：BOY

■ puts函数：将一个字符串(以'\0'结束的字符序列)输出到终端

- 用`puts`函数输出的字符串中可以包含转义字符。
- 用`puts`输出时将字符串结束标志'\0'转换成'\n'，即输出完字符串后换行
- `puts(str)`相当于`printf("%s\n", str)`

■ gets函数：从终端输入一个字符串到字符数组

- 该函数值是字符数组的起始地址
- `gets(str)`不完全相当于`scanf("%s\n", str)`
- 若从键盘输入 `My Computer`，将输入的字符串" `My Computer`"送给字符数组 `str` (11个字符外加1个'\0')，回车被消耗掉，输入缓冲区当前位置指向下一个字符

```
1. #include <stdio.h>
2. int main(int argc, char **argv) {
3.     char str1[]={"Info\nTuring"};
4.     char str2[100];
5.     gets(str2);
6.     puts(str1);
7.     puts(str2);
8. }
```

输入：RUC

输出：Info
Turing
RUC

■ puts函数：将一个字符串(以'\0'结束的字符序列)输出到终端

- 用`puts`函数输出的字符串中可以包含转义字符。
- 用`puts`输出时将字符串结束标志'\0'转换成'\n'，即输出完字符串后换行
- `puts(str)`相当于`printf("%s\n", str)`

■ gets函数：从终端输入一个字符串到字符数组

- 该函数值是字符数组的起始地址
- `gets(str)`不完全相当于`scanf("%s\n", str)`
- 若从键盘输入 `My Computer`，将输入的字符串" `My Computer`"送给字符数组 `str` (11个字符外加1个'\0')，回车被消耗掉，输入缓冲区当前位置指向下一个字符

```
1. #include <stdio.h>
2. int main(int argc, char **argv) {
3.     char str1[]={"Info\nTuring"};
4.     char str2[100];
5.     gets(str2);
6.     puts(str1);
7.     puts(str2);
8. }
```

输入：RUC

输出：Info
Turing
RUC

8.1.2 语法1——程序设计基本概念

■ 格式化输入背后的原理

■ 输入缓冲区：存储用户键盘输入的字符数据

- 当用户敲击Enter键，数据进入输入缓冲区
- 若输入缓冲区中的数据不够满足读入操作的需求，则继续等待用户输入
- 输入缓冲区中的残存数据会影响下一次数据读入操作

■ getchar：获得输入缓冲区的当前字符

■ gets：从输入缓冲区读入字符直至遇到换行符

终端：100 □ □ a ↵

输入缓冲区：\x31\x30\x30\x20\x20\x61\x0A

■ scanf：按照格式声明指定的格式从输入缓冲区中读取数据

■ 依次匹配格式声明和输入缓冲区

- %c：获得输入缓冲区的当前字符，即使是一个分隔符
- %s/%d/%f：跳过前导分隔符，读取输入缓冲区内容，直至遇到第一个分隔符结束，不消耗后导分隔符

■ 将输入缓冲区的字符数据转换成格式化字符串指定类型的数据

■ 若遇到格式声明中的分隔符（\n、\t、空格），则消耗缓冲区当前位置往后所有分隔符

■ 当输入缓冲区中的数据不满足格式声明要求时，立即终止本次数据读取

终端：100 □ □ a ↵

输入缓冲区：\x31\x30\x30\x20\x20\x61\x0A

格式声明："%d □ %c"

终端：a □ □ 100 ↵

输入缓冲区：\x61\x20\x20\x31\x30\x30\x0A

格式声明："%c □ %d"

8.1.2 语法1——程序设计基本概念

■ 格式化输入背后的原理

以下程序在给定的输入下，输出什么？

```
int x; char y;  
scanf("%d%c", &x, &y);  
printf("%d %c", x, y);
```

input: 1000.1↵

output: 1000 .

```
char x, y, z;  
scanf("%c%c%c", &x, &y, &z);  
printf("%c%c%c", x, y, z);
```

input: a b c ↵

output: a b c

```
char x[100]; char y;  
scanf("%s%c", &x, &y);  
printf("%s%c", x, y);
```

input: abcd↵

output: abcd

```
unsigned int x, y;  
scanf("%d%d", &x, &y);  
printf("%d %d", x, y);
```

input: 12345678987654321 ↵
12↵

output:
1653732529 12

```
char x[100]; int y = 0;  
scanf("%s,%d", &x, &y);  
printf("%s %d", x, y);
```

input: 1234,1↵

output: 1234,1 0

input: 1234 ↵,1↵

output: 1234 0

```
int x=0, y=0, z=0;  
scanf("%d,%d", &x, &y);  
scanf("%d", &z);  
printf("%d %d %d\n", x, y, z);
```

input: 1,2 ↵3↵

output: 1 2 3

input: 1 ↵2 ↵3↵

output: 1 0 2

```
char x[100], y;  
gets(x);  
y = getchar();  
printf("%s %c", x, y);
```

input: abc ↵d↵

e↵

output: abc d e

note:

(12345678987654321)₁₀
<==>10进制转2进制
(001010111101110001010100
011000101001000111110100
10110001)₂
=|=>截断
(011000101001000111110100
10110001)₂
<==>2进制转10进制
(1653732529)₁₀

8.1.3 语法2——程序控制语句

- 关系运算/逻辑运算/条件运算

- 关系运算：将两个数值进行比较，判断其比较的结果是否符合给定的条件

- 关系表达式只能表达一些简单的条件
- 每个判断只是对一个条件进行测试

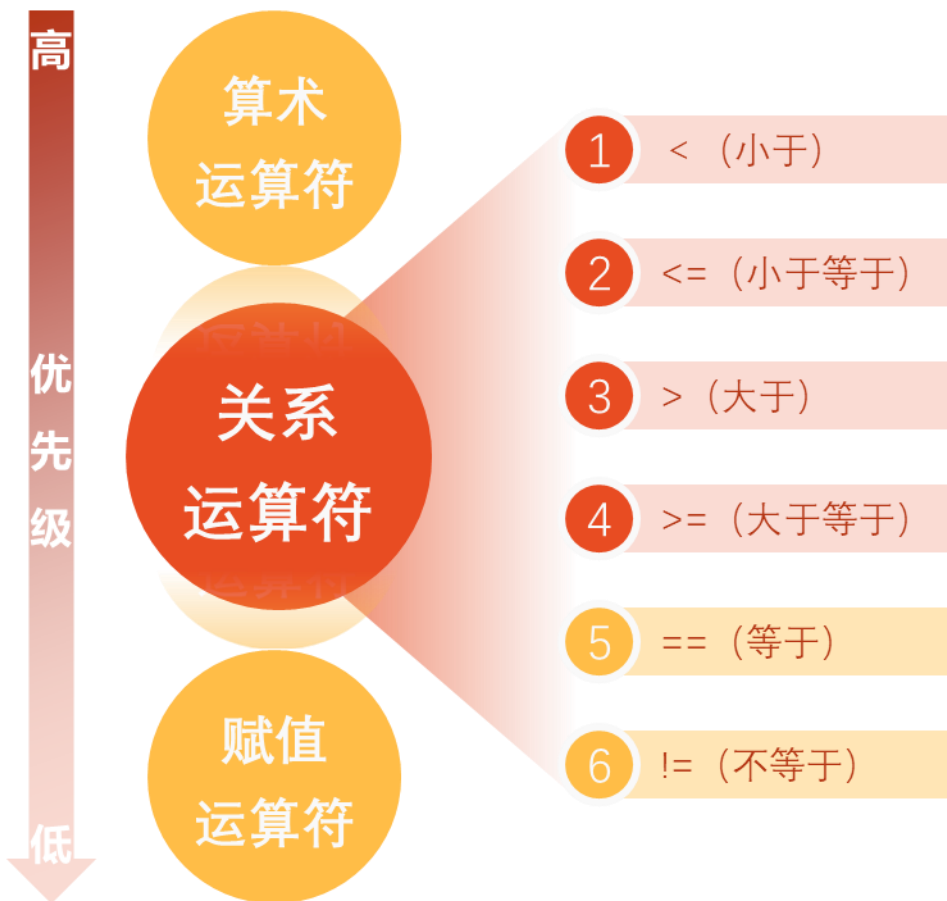
- 逻辑运算：通过逻辑运算符把简单的条件组合起来，能够形成更加复杂的条件

- 例： $10 > y > 5$ 的逻辑表达式
- 例： $x < -10$ 或者 $x > 0$ 的逻辑表达式

- 条件运算：C语言中唯一的一个三目运行符，根据不同的结果来决定计算哪个子表达式

8.1.3 语法2——程序控制语句

■ 关系运算与关系表达式



- 前 4 种关系运算符的优先级别相同，后 2 种也相同。前 4 种高于后 2 种。
- 关系运算符的优先级低于算术运算符。
- 关系运算符的优先级高于赋值运算符。

$c > a + b$ 等效于 $c > (a + b)$
(关系运算符的优先级低于算术运算符)

$a > b == c$ 等效于 $(a > b) == c$
(大于运算符 $>$ 的优先级高于相等运算符 $==$)

$a == b < c$ 等效于 $a == (b < c)$
(小于运算符 $<$ 的优先级高于相等运算符 $==$)

$a = b > c$ 等效于 $a = (b > c)$
(关系运算符的优先级高于赋值运算符)

8.1.3 语法2——程序控制语句

■ 关系运算与关系表达式

- 用关系运算符将两个数值或数值表达式连接起来的式子，称为关系表达式
- 关系表达式的值是一个逻辑值，即“真”或“假”
- 在C的逻辑运算中，以“1”代表“真”，以“0”代表“假”

若 $a=3$, $b=2$, $c=1$, 则:

$d=a>b$, 由于 $a>b$ 为真, 因此关系表达式 $a>b$ 的值为1, 所以赋值后 d 的值为1。

$f=a>b>c$, 则 f 的值为0。因为“ $>$ ”运算符是自左至右的结合方向, 先执行“ $a>b$ ”得值为1, 再执行关系运算“ $1>c$ ”, 得值0, 赋给 f , 所以 f 的值为0

8.1.3 语法2——程序控制语句

■ 逻辑运算符与逻辑表达式

运算符	含义	举例	说明
!	逻辑非(NOT)	!a	如果a为假，则!a为真;如果a为真，则!a为假
&&	逻辑与(AND)	a && b	如果a和b都为真，则结果为真，否则为假
	逻辑或(OR)	a b	如果a和b有一个以上为真，则结果为真，二者都为假时，结果为假

- “&&”和“||”是双目运算符，要求有两个运算对象(操作数)；“!”是单目运算符，只要有一个运算对象
- 优先次序：!(非)→&&(与)→||(或)，即“!”为三者中最高的；逻辑运算符中的“&&”和“||”低于关系运算符，“!”高于算术运算符
- 逻辑运算结果不是0就是1，不可能是其他数值。而在逻辑表达式中作为参加逻辑运算的运算对象可以是0(“假”)或任何非0的数值(按“真”对待)

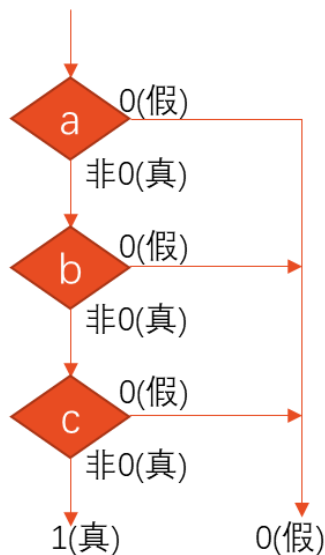
a	b	!a	!b	a && b	a b
真 (非0)	真 (非0)	假 (0)	假 (0)	真 (1)	真 (1)
真 (非0)	假 (0)	假 (0)	真 (1)	假 (0)	真 (1)
假 (0)	真 (非0)	真 (1)	假 (0)	假 (0)	真 (1)
假 (0)	假 (0)	真 (1)	真 (1)	假 (0)	假 (0)

8.1.3 语法2——程序控制语句

■ 逻辑运算符与逻辑表达式

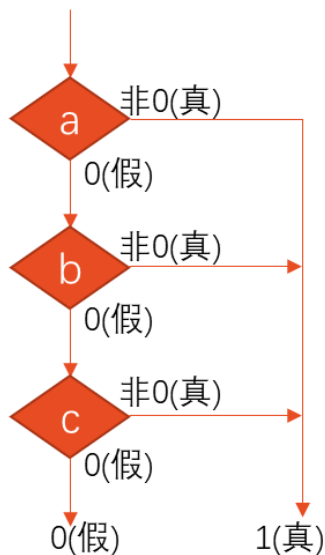
- 在逻辑表达式的求解中，并不是所有的逻辑运算符都被执行，只是在必须执行下一个逻辑运算符才能求出表达式的解时，才执行该运算符。

- $a \ \&\& \ b \ \&\& \ c$ 。只有a为真(非0)时，才需要判别b的值。只有当a和b都为真时才需要判别c的值。



逻辑
表达式

- $a \ || \ b \ || \ c$ 。只要a为真(非0)，就不必判断b和c。只有a为假，才判别b。a和b都为假才判别c。



8.1.3 语法2——程序控制语句

■ 逻辑运算符与逻辑表达式

■ 示例：下列代码在特定输入下的输出

```
1. int main()
2. {
3.     int a, b, c;
4.     int x, y, z;
5.     scanf("%d%d%d", &a, &b, &c);

6.     x = a; y = b; z = c;
7.     if (!x && y++) z--;
8.     printf("x=%d y=%d z=%d\n", x, y, z);

9.     x = a; y = b; z = c;
10.    if (!x && ++y) z--;
11.    printf("x=%d y=%d z=%d\n", x, y, z);

12.    x = a; y = b; z = c;
13.    if (!x || y++) z--;
14.    printf("x=%d y=%d z=%d\n", x, y, z);

15.    x = a; y = b; z = c;
16.    if (!x || ++y) z--;
17.    printf("x=%d y=%d z=%d\n", x, y, z);
18.}
```

输入1:

0 0 0

输出1:

x=0 y=1 z=0

x=0 y=1 z=-1

x=0 y=0 z=-1

x=0 y=0 z=-1

输入2:

0 1 0

输出2:

x=0 y=2 z=-1

x=0 y=2 z=-1

x=0 y=1 z=-1

x=0 y=1 z=-1

输入3:

1 0 0

输出3:

x=1 y=0 z=0

x=1 y=0 z=0

x=1 y=1 z=0

x=1 y=1 z=-1

输入4:

1 1 0

输出4:

x=1 y=1 z=0

x=1 y=1 z=0

x=1 y=2 z=-1

x=1 y=2 z=-1

8.1.3 语法2——程序控制语句

■ 条件运算符与条件表达式

```
if (a>b)
```

```
    max=a;
```

```
else
```

```
    max=b;
```



```
max=(a>b) ? a : b;
```

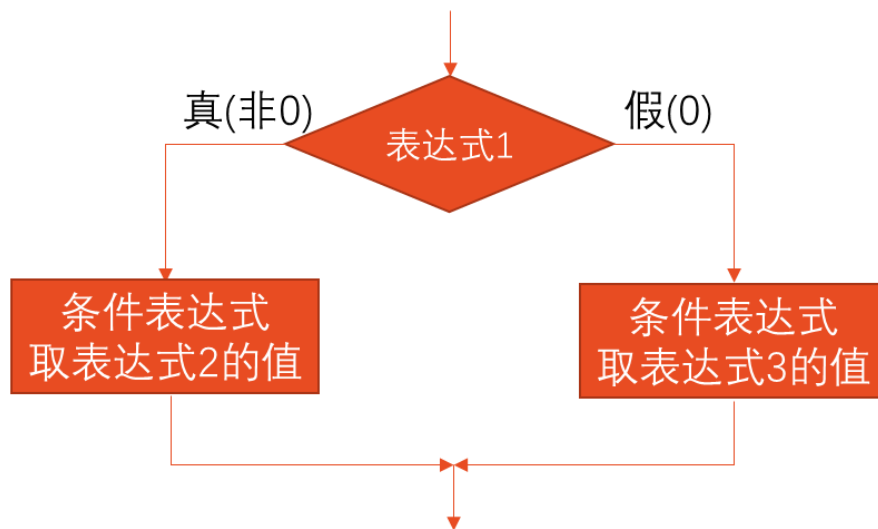
或

```
a>b ? (max=a) : (max=b);  
//表达式2和表达式3是赋值表达式
```

表达式1 ? 表达式2 : 表达式3

条件运算符由两个符号(?)和(:)组成, 必须一起使用。要求有3个操作对象, 称为三目(元)运算符, 它是C语言中唯一的一个三目运算符。

条件运算符的执行顺序: 先求解表达式1, 若为非0(真)则求解表达式2, 此时表达式2的值就作为整个条件表达式的值。若表达式1的值为0(假), 则求解表达式3, 表达式3的值就是整个条件表达式的值。



8.1.3 语法2——程序控制语句

■ 条件运算符与条件表达式

- 示例：输入一个字符，判别它是否为大写字母，如果是，将它转换成小写字母；如果不是，不转换。然后输出最后得到的字符

```
1. #include <stdio.h>

2. int main()
3. {
4.     char ch;
5.     scanf("%c", &ch);
6.     ch = (ch >= 'A' && ch <= 'Z') ? (ch + 32) : ch;
7.     printf("%c\n", ch);
8.     return 0;
9. }
```



条件表达式“(ch>='A'&&ch<='Z')?(ch+32):ch”的作用是：如果字符变量ch的值为大写字母，则条件表达式的值为(ch+32)，即相应的小写字母，32是小写字母和大写字母ASCII的差值。如果ch的值不是大写字母，则条件表达式的值为ch，即不进行转换。

8.1.3 语法2——程序控制语句

■ 选择语句：

- if () ... else ...：两个分支
- switch ()：多个分支

■ 循环语句：

- for () ...
- while () ...
- do ... while ()

■ 改变执行流程语句：

- 结束本次循环语句：continue
- 终止执行switch或循环语句：break
- 转向语句：goto（慎用）
- 从函数返回语句：return

if语句

■ 示例：写一程序，判断某一年是否为闰年

year被4整除				假	
真		假		假	
year被100整除				假	
真		假		leap=0	
year被400整除				leap=1	
真		假		leap=1	
leap=1		leap=0		leap=1	
真				假	
leap				假	
输出“闰年”				输出“非闰年”	

switch语句

- 示例：要求按照考试成绩的等级输出百分制分数段，A等为90分以上，B等为80～89分，C等为70～79分，D等为60～69分，E等为60分以下。成绩的等级由键盘输入。

```
1. #include <stdio.h>
2. int main() {
3.     char grade;
4.     scanf("%c",&grade);
5.     printf("Your score:");
6.     switch(grade) {
7.         case 'A': printf("90~100\n");break;
8.         case 'B': printf("80~89\n");break;
9.         case 'C': printf("70~79\n");break;
10.        case 'D': printf("70~79\n");break;
11.        case 'E': printf("<60\n");break;
12.        default:  printf("enter data error!\n");
13.    }
14.    return 0;
15.}
```



等级grade定义为字符变量，从键盘输入一个大写字母，赋给变量grade，switch得到grade的值并把它和各case中给定的值('A','B','C','D'之一)相比较，如果和其中之一相同(称为匹配)，则执行该case后面的语句(即printf语句)。如果输入的字符与'A','B','C','D'都不相同，就执行default后面的语句，**注意在每个case后面后的语句中，最后都有一个break语句，它的作用是使流程转到switch语句的末尾(即右花括号处)。**

switch语句

思考：对于右侧代码，
输入为'A', 'B', 'C', 'D', 'E'
时，输出分别是什么？

```
1. #include <stdio.h>
2. int main() {
3.     char grade;
4.     scanf("%c",&grade); printf("Your score:");
5.     switch(grade) {
6.         case 'A': { printf("90~100\n");break; }
7.         case 'B': printf("80~89\n");
8.         case 'C': printf("70~79\n");break;
9.         case 'D': case 'E': printf("<60\n");break;
10.        default:  printf("enter data error!\n");
11.    }
12.}
```

switch(表达式)

{

case 常量1 : 语句1

case 常量2 : 语句2

⋮ ⋮ ⋮

case 常量n : 语句n

default : 语句n+1

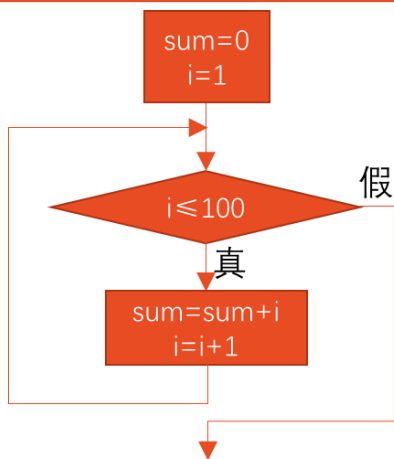
}

- (1) 括号内的“表达式”，其值的类型应为整数类型(包括字符型)。
- (2) 花括号内是一个复合语句，包含多个case开头的语句行和最多一个default开头的语句行。case后面跟一个常量(或常量表达式)，它们和default都是起标号作用，用来标志一个位置。
- (3) 执行switch语句时，先计算switch后面的“表达式”的值，然后将它与各case标号比较，若与某一个case标号中的常量相同，控制流就转到匹配的case标号后面的语句。若没有任何匹配，控制流转去执行default标号后面的语句；若没有default标号，则不执行任何语句。
- (4) 在case子句中虽然包含了一个以上执行语句，但可以不必用花括号括起来，会自动顺序执行本case标号后面所有的语句。当然加上花括号也可以。
- (5) 各个case标号出现次序不影响执行结果，但每一个case常量必须互不相同；否则就会出现互相矛盾的现象
- (6) case标号只起标记的作用。在执行switch语句时，根据switch表达式的值找到匹配的入口标号，在执行完一个case标号后面的语句后，就从此标号开始执行下去，不再进行判断。因此，一般情况下，在执行一个case子句后，应当用break语句使流程跳出switch结构。
- (7) 多个case标号可以共用一组执行语句。

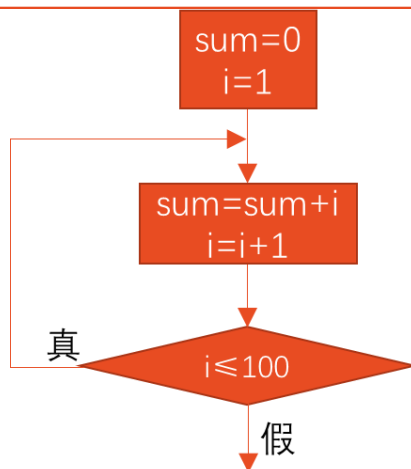
while/do...while/for语句

示例：求 $1+2+3+\dots+100$ ，即 $\sum_{n=1}^{100} n$

```
1. #include<stdio.h>
2. int main()
3. {
4.     int i=1,sum=0;
5.     while(i<=100)
6.     {
7.         sum=sum+i;
8.         i++;
9.     }
10.    printf("sum=%d\n",sum);
11.    return 0;
12.}
```

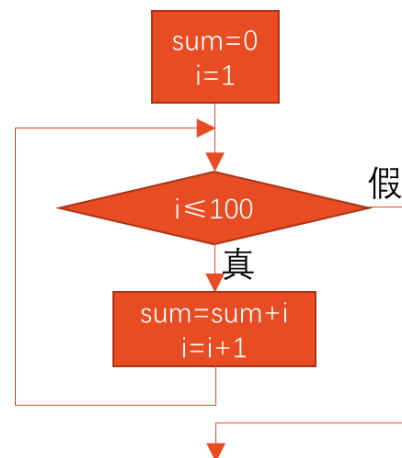


```
1. #include <stdio.h>
2. int main()
3. {
4.     int i=1,sum=0;
5.     do
6.     {
7.         sum=sum+i;
8.         i++;
9.     }while(i<=100);
10.    printf("sum=%d\n",sum);
11.    return 0;
12.}
```



```
1. #include <stdio.h>
2. int main()
3. {
4.     int i,sum=0;
5.     for (i = 1; i <= 100; i++)
6.     {
7.         sum=sum+i;
8.     }
9. }
10. printf("sum=%d\n",sum);
11. return 0;
12.}
```

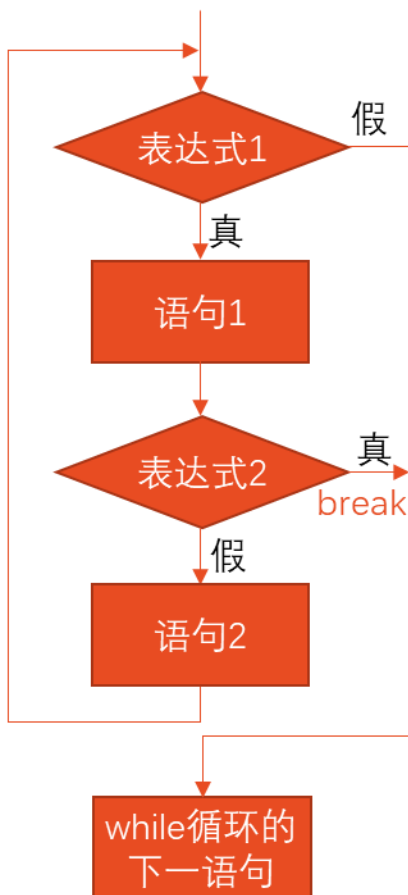
可简写为 `for (i = 1, sum = 0; i <= 100; sum+=i, i++);`



用break和continue语句改变循环流程

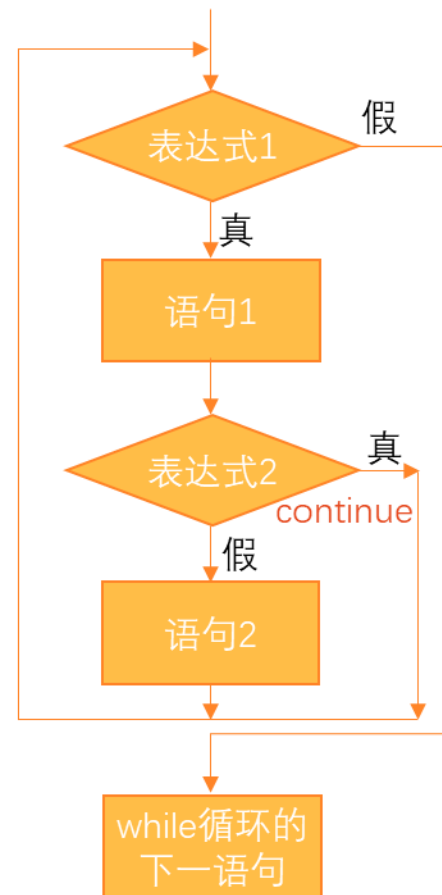
break;

```
while(表达式1)
{
    语句1
    if(表达式2) break;
    语句2
}
```



continue;

```
while(表达式1)
{
    语句1
    if(表达式2) continue;
    语句2
}
```



continue语句只结束本次循环，而非终止整个循环。break语句结束整个循环，不再判断执行循环的条件是否成立。

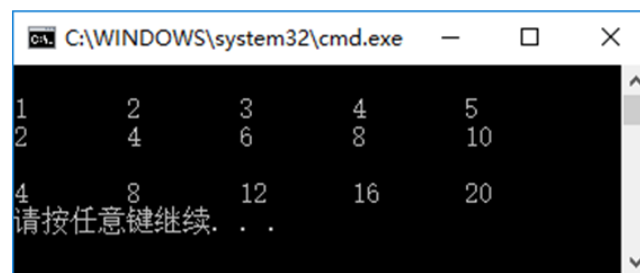
用break和continue语句改变循环流程

■ 示例：输出以下4×5的矩阵

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20

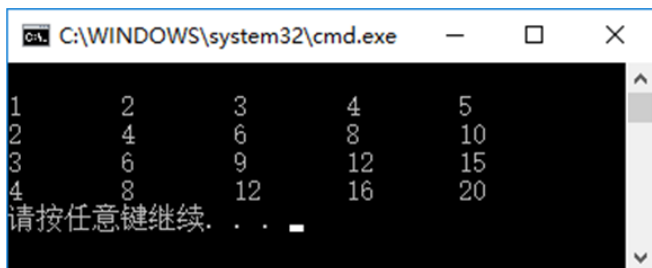
if (i==3 && j==1) break;

```
1. #include <stdio.h>
2. int main()
3. {
4.     int i,j,n=0;
5.     for(i=1;i<=4;i++)
6.         for(j=1;j<=5;j++,n++)    //n: 累计输出数据个数
7.         {
8.             if(n%5==0) printf("\n"); //输出5个数据后换行
9.             printf("%d\t",i*j);
10.        }
11.    printf("\n");
12.    return 0;
13.}
```

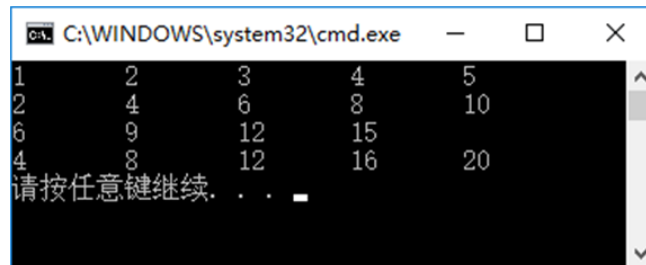


```
C:\WINDOWS\system32\cmd.exe
1      2      3      4      5
2      4      6      8      10
3      6      9      12     15
4      8      12     16     20
请按任意键继续. . .
```

if (i==3 && j==1) continue;



```
C:\WINDOWS\system32\cmd.exe
1      2      3      4      5
2      4      6      8      10
3      6      9      12     15
4      8      12     16     20
请按任意键继续. . .
```



```
C:\WINDOWS\system32\cmd.exe
1      2      3      4      5
2      4      6      8      10
3      6      9      12     15
4      8      12     16     20
请按任意键继续. . .
```


8.1.4 算法1——穷举法

- 基本思想：将问题的所有可能的答案一一列举，然后根据条件判断此答案是否合适，保留合适的，丢弃不合适的
- 使用枚举算法解题的基本思路：
 - 确定枚举对象、枚举范围和判定条件
 - 逐一枚举可能的解，验证每个解是否是问题的解
- 枚举算法一般按照如下3个步骤进行
 - 题解的可能范围，不能遗漏任何一个真正解，也要避免有重复
 - 判断是否是真正解的方法
 - 使可能解的范围降至最小，以便提高解决问题的效率

8.1.4 算法1——穷举法

■ 用穷举法解决逻辑问题

■ 示例1：谁做的好事

人大附中有四位同学中的一位做了好事，不留名，表扬信来了之后，校长问这四位是谁做的好事。

A说：不是我。

B说：是C。

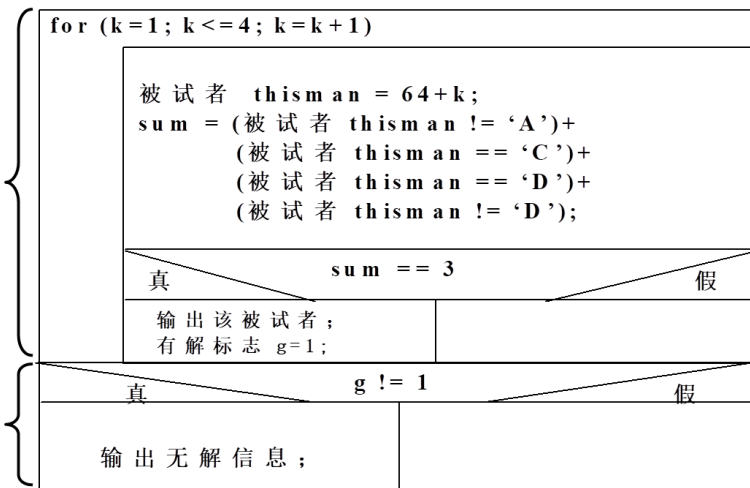
C说：是D。

D说：他胡说。

已知三个人说的是真话，一个人说的是假话。现在要根据这些信息，找出做了好事的人

第一块 循环结构

第二块 分支结构



```
1. #include <stdio.h>
2. int main(int argc, char **argv) {
3.     int k=0, sum=0, flag=0;
4.     char thisman = ' ';
5.     for (k=1; k<=4; k++) {
6.         thisman = 64+k; //A的ASCII码是65
7.         sum = (thisman!='A') + (thisman=='C') + (thisman=='D') + (thisman!='D');
8.         if (sum == 3) {
9.             printf("%c did the good thing.\n", thisman);
10.            flag = 1;
11.        }
12.    }
13.    if (flag != 1) printf("Can't find who did the good thing.\n");
14.}
```

8.1.4 算法1——穷举法

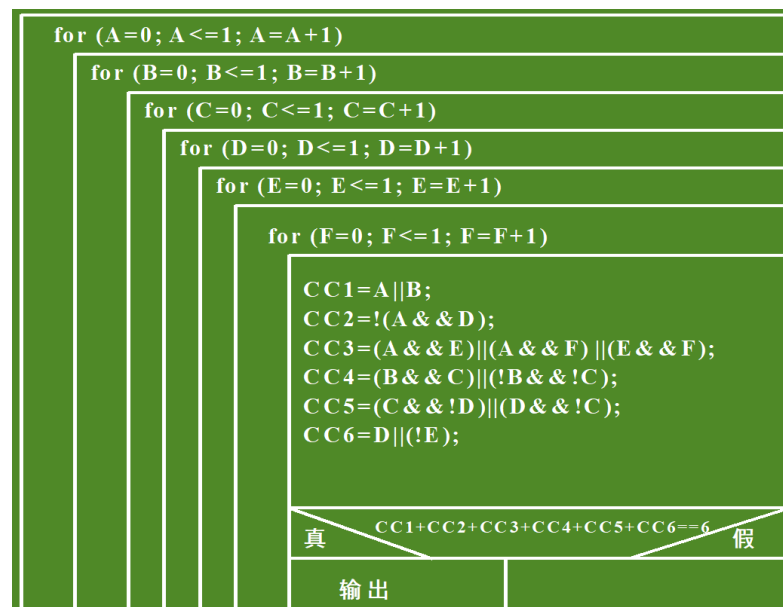
■ 用穷举法解决逻辑问题

■ 示例2：破案

某地刑侦大队对涉及六个嫌疑人的一桩疑案进行分析：

- A、B 至少有一人作案；
- A、E、F 三人中至少有两人参与作案；
- A、D 不可能是同案犯；
- B、C 或同时作案，或与本案无关；
- C、D 中有且仅有一人作案；
- 如果 D 没有参与作案，则 E 也不可能参与作案

试编一程序，将作案人找出来



```
1. #include <stdio.h>
2. int main(int argc, char **argv) {
3.     int cc1, cc2, cc3, cc4, cc5, cc6;
4.     int A, B, C, D, E, F;
5.     for (A=0; A<=1; A++)
6.         for (A=0; A<=1; A++)
7.             for (A=0; A<=1; A++)
8.                 for (A=0; A<=1; A++)
9.                     for (A=0; A<=1; A++)
10.                        for (A=0; A<=1; A++)
11.                           {
12.                               cc1 = A||B; cc2 = !(A&&D); cc3 = (A&&E)|| (A&&F) || (E&&F);
13.                               cc4 = (B&&C)|| (!B&&!C); cc5 = (C&&!D)|| (!C&&D); cc6 = D||!E;
14.                               if (cc1+cc2+cc3+cc4+cc5+cc6 == 6) {
15.                                   printf("A:%d B:%d C:%d D:%d E:%d F:%d\n", A, B, C, D, E, F);
16.                                   break;
17.                               }
18. }
```

8.1.4 算法1——穷举法

■ 用穷举法解决数值问题

■ 示例3：百钱买百鸡

公元前五世纪，我国古代数学家张丘建在《算经》一书中提出了“百鸡问题”：鸡翁一值钱五，鸡母一值钱三，鸡雏三值钱一。百钱买百鸡，问鸡翁、母、雏各几何？

解题的基本思路：用变量 n_cocks , n_hens , n_chicks 分别表示鸡翁、母鸡、雏鸡的数量

- $n_cocks + n_hens + n_chicks = 100$ (百鸡)
- $5 * n_cocks + 3 * n_hens + n_chicks / 3 = 100$ (百钱)

■ 示例4：搬砖问题

36块砖，36人搬；男搬4，女搬3，两个小孩抬一砖。要求一次全搬完，问男、女、小孩各若干？

解题的基本思路：

- $4 * men + 3 * women + children / 2 = 36$
- $men + women + children = 36$
- men取值范围：0~9
- women取值范围：0~12
- children取值范围：0~36

8.1.5 数据结构1——数组与自定义数据类型

■ 一维数组

定义一维数组

类型说明符 数组名[常量表达式]

```
int a[10];
```

整型数组，即数组中的元素均为整型

数组名为a

数组包含10个整型元素

(1) 数组名的命名规则和变量名相同，遵循标识符命名规则。

a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8] a[9]

相当于定义了10个简单的整型变量

(2) 在定义数组时，需要指定数组中元素的个数，方括号中的常量表达式用来表示元素的个数，即数组长度。

注意

• 数组元素的下标从0开始，用“int a[10];”定义数组，则最大下标值为9，不存在数组元素a[10]

(3) 常量表达式中可以包括常量和符号常量，不能包含变量。

引用一维数组

数组名[下标]

只能引用数组元素而不能一次整体调用整个数组全部元素的值。

数组元素与一个简单变量的地位和作用相似。

“下标”可以是整型常量或整型表达式。

注意

• 定义数组时用到的“数组名[常量表达式]”和引用数组元素时用的“数组名[下标]”在形式上相同，但含义不同。

```
int a[10];  
//前面有int这是定义数组,指定数组包含10个元素  
  
t=a[6];  
//这里的a[6]表示引用a数组中序号为6的元素
```

一维数组的初始化

为了使程序简洁，常在定义数组的同时给各数组元素赋值，这称为数组的初始化。

(1) 在定义数组时对全部数组元素赋予初值。

```
int a[10]={0,1,2,3,4,5,6,7,8,9};
```

将数组中各元素的初值顺序放在一对花括号内，数据间用逗号分隔。花括号内的数据就称为“初始化列表”。

(2) 可以只给数组中的一部分元素赋值。

```
int a[10]={0,1,2,3,4};
```

定义a数组有10个元素，但花括号内只提供5个初值，这表示只给前面5个元素赋初值，系统自动给后5个元素赋初值为0。

(3) 给数组中全部元素赋初值为0。

```
int a[10]={0,0,0,0,0,0,0,0,0,0};
```

或

```
int a[10]={0}; //未赋值的部分元素自动设定为0
```

(4) 在对全部数组元素赋初值时，由于数据的个数已经确定，因此可以不指定数组长度。

```
int a[5]={1,2,3,4,5};
```

或

```
int a[]={1,2,3,4,5};
```

但是，如果数组长度与提供初值的个数不相同，则方括号中的数组长度不能省略。

一维数组的存储

C语言中，一维数组在内存中是一块连续的区域，按下标大小依次存储。

```
float a[4]
```

a₀

a₁

a₂

a₃

2000

a[0]

2004

a[1]

2008

a[2]

2012

a[3]

8.1.5 数据结构1——数组与自定义数据类型

二维数组

小例子

有3个小分队，每队有6名队员，要把这些队员的工资用数组保存起来以备查。

	队员1	队员2	队员3	队员4	队员5	队员6
第1分队	2456	1847	1243	1600	2346	2757
第2分队	3045	2018	1725	2020	2458	1436
第3分队	1427	1175	1046	1976	1477	2018

定义二维数组

类型说明符 数组名[常量表达式][常量表达式]



`float a[3][4], b[5][10];`
//定义a为3×4(3行4列)的数组，b为5×10(5行10列)的数组



`float a[3, 4], b[5, 10];` //在一对方括号内不能写两个下标

`float pay[3][6];`

float型二维数组

数组名为pay

数组第二维有6个元素

数组第一维有3个元素

二维数组可被看作一种特殊的一维数组：它的元素又是一个一维数组。

例如，`float a[3][4];`可以把a看作一个一维数组，它有3个元素：`a[0]`，`a[1]`，`a[2]`，每个元素又是一个包含4个元素的一维数组：

`a[0]` —— `a[0][0]` `a[0][1]` `a[0][2]` `a[0][3]`

`a[1]` —— `a[1][0]` `a[1][1]` `a[1][2]` `a[1][3]`

`a[2]` —— `a[2][0]` `a[2][1]` `a[2][2]` `a[2][3]`

引用二维数组元素

数组名[下标][下标]

注意

- 在引用数组元素时，下标值应在已定义的数组大小的范围内。

“下标”可以是整型常量或整型表达式。

数组元素可以出现在表达式中，也可以被赋值，如：`b[1][2]=a[2][3]/2;`

`int a[3][4];` //定义a为3×4的二维数组

`a[3][4]=3;` //不存在`a[3][4]`元素
//数组a可用的“行下标”的范围为0~2，“列下标”的范围为0~3



- 严格区分在定义数组时用的`a[3][4]`和引用元素时的`a[3][4]`的区别。
- 前者用`a[3][4]`来定义数组维数和各维大小
- 后者`a[3][4]`中的3和4是数组元素的下标值，`a[3][4]`代表行序号为3、列序号为4的元素（行序号和列序号均从0起算）。

二维数组的存储

C语言中，二维数组中元素排列的顺序是按行存放的。

`float a[3][4]`



注意

- 用矩阵形式（如3行4列形式）表示二维数组，是逻辑上的概念，能形象地表示出行列关系。而在内存中，各元素是连续存放的，不是二维的，是线性的。

2000
2004
2008
2012
2016
2020
2024
2028
2032
2036
2040
2044

`a[0][0]`
`a[0][1]`
`a[0][2]`
`a[0][3]`
`a[1][0]`
`a[1][1]`
`a[1][2]`
`a[1][3]`
`a[2][0]`
`a[2][1]`
`a[2][2]`
`a[2][3]`

8.1.5 数据结构1——数组与自定义数据类型

■ 二维数组

二维数组的初始化

可以用“初始化列表”对二维数组初始化。

(1)分行给二维数组赋初值。（最清楚直观）

```
int a[3][4]={{1,2,3,4},{5,6,7,8},{9,10,11,12}};
```

(2)可以将所有数据写在一个花括号内，按数组元素在内存中的排列顺序对各元素赋初值。

```
int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

(3)可以对部分元素赋初值。

```
int a[3][4]={{1},{5},{9}}; ①
```

```
int a[3][4]={{1},{0,6},{0,0,11}}; ②
```

```
int a[3][4]={{1},{5,6}}; ③
```

```
int a[3][4]={{1},{},{9}}; ④
```

①	②	③	④
1 0 0 0	1 0 0 0	1 0 0 0	1 0 0 0
5 0 0 0	0 6 0 0	5 6 0 0	0 0 0 0
9 0 0 0	0 0 11 0	0 0 0 0	9 0 0 0

(4)如果对全部元素都赋初值(即提供全部初始数据)，则定义数组时对第1维的长度可以不指定，但第2维的长度不能省。

```
int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

≡

```
int a[][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

在定义时也可以只对部分元素赋初值而省略第1维的长度，但应分行赋初值。

```
int a[][4]={{0,0,3},{},{0,10}};
```

8.1.5 数据结构1——数组与自定义数据类型

■ 多维数组

```
float a[2][3][4];    //定义三维数组a，它有2页，3行，4列
```

float a[2, 3, 4];在内存中的排列顺序为：

a[0][0][0] → a[0][0][1] → a[0][0][2] → a[0][0][3] →

a[0][1][0] → a[0][1][1] → a[0][1][2] → a[0][1][3] →

a[0][2][0] → a[0][2][1] → a[0][2][2] → a[0][2][3] →

a[1][0][0] → a[1][0][1] → a[1][0][2] → a[1][0][3] →

a[1][1][0] → a[1][1][1] → a[1][1][2] → a[1][1][3] →

a[1][2][0] → a[1][2][1] → a[1][2][2] → a[1][2][3]

8.1.5 数据结构1——数组与自定义数据类型

■ 字符数组

```
1. #include <stdio.h>
2. int main()
3. {
4.     char c[15]={'I',' ','a','m',' ',' ','a',' ',' ',
5.                's','t','u','d','e','n','t','.'};
6.     int i;
7.     for(i=0;i<15;i++)
8.         printf("%c",c[i]);
9.     printf("\n");
10.    return 0;
11.}
```

输出一个已知的字符串

```
1. #include <stdio.h>
2. int main()
3. {
4.     char diamond[][5]={{' ',' ',' ','*'},
5.                          {' ','*',' ',' ','*'},
6.                          {'*',' ',' ',' ','*'},
7.                          {' ','*',' ',' ','*'},
8.                          {' ',' ',' ','*'}};
9.     int i,j;
10.    for (i=0;i<5;i++)
11.    {
12.        for (j=0;j<5;j++)
13.            printf("%c",diamond[i][j]);
14.        printf("\n");
15.    }
16.    return 0;
17.}
```

输出一个菱形图

用来存放字符数据的数组是**字符数组**。在字符数组中的一个元素内存放一个字符。

```
char c[10];
c[0]='I'; c[1]=' '; c[2]='a'; c[3]='m'; c[4]=' '; c[5]='h'; c[6]='a'; c[7]='p'; c[8]='p'; c[9]='y';
```

c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]	c[8]	c[9]
I		a	m		h	a	p	p	y

由于字符型数据是以整数形式(ASCII代码)存放的，故也可以用整型数组来存放字符数据。

```
int c[10];
c[0]='a';    //合法，但浪费存储空间
```

对字符数组初始化，最容易理解的方式是用“初始化列表”，把各个字符依次赋给数组中各元素。

```
char c[10]={'I',' ','a','m',' ','h','a','p','p','y'};    //把10个字符依次赋给c[0] ~ c[9]这10个元素
```

如果在定义字符数组时不进行初始化，则数组中各元素的值是**不可预料**的。

如果花括号中提供的初值个数（即字符个数）大于数组长度，则出现语法错误。

如果初值个数小于数组长度，则只将这些字符赋给数组中前面那些元素，其余的元素自动定为空字符(即'\0')。

```
char c[10]={'c',' ','p','r','o','g','r','a','m'};
```

c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]	c[8]	c[9]
c		p	r	o	g	r	a	m	\0

如果提供的初值个数与预定的数组长度相同，定义时可以省略数组长度，系统会自动根据初值个数确定长度。

```
char c[]={ 'I',' ','a','m',' ','h','a','p','p','y'};    //数组c的长度自动定为10
```

也可以定义和初始化一个二维字符数组。

```
char diamond[5][5]={{ ' ',' ','*'},{' ','*',' ','*'},{'*',' ',' ','*'},{' ','*',' ','*'},{' ',' ','*'}};
```

```
*
* *
*  *
* *
*
```

8.1.5 数据结构1——数组与自定义数据类型

■ 字符串

(1) 逐个字符输入输出。用格式符 “%c” 输入或输出一个字符。

(2) 将整个字符串一次输入或输出。用 “%s” 格式符，意思是对字符串(string)的输入输出。

```
1. #include <stdio.h>
2. int main()
3. {
4.     char c[15]={'I',' ','a','m',' ',' ','a',' ',' ',
5.                's','t','u','d','e','n','t','.'};
6.     int i;
7.     for(i=0;i<15;i++)
8.         printf("%c",c[i]);
9.     printf("\n");
10.    return 0;
11.}
```

(1) 输出的字符中不包括结束符'\0'。

(2) 用“%s”格式符输出字符串时，printf函数中的输出项是字符数组名，而不是数组元素名。

(3) 如果数组长度大于字符串的实际长度，也只输出到遇'\0'结束。

(4) 如果一个字符数组中包含一个以上'\0'，则遇第一个'\0'时输出就结束。

在C语言中，是将字符串作为字符数组来处理的。

在实际工作中，人们关心的往往是字符串的有效长度而不是字符数组的长度。

为了测定字符串的实际长度，C语言规定了一个“字符串结束标志”，以字符'\0'作为结束标志。

“C program” 字符串是存放在一维数组中的，占10个字节，字符占9个字节，最后一个字节'\0'是由系统自动加上的

注意

- C系统在用字符数组存储字符串常量时会自动加一个'\0'作为结束符。
- 在定义字符数组时应估计实际字符串长度，保证数组长度始终大于字符串实际长度。
- 如果在一个字符数组中先后存放多个不同长度的字符串，则应使数组长度大于最长的字符串的长度。

```
printf("How do you do?\n");
```

在向内存中存储时，系统自动在最后一个字符'\n'的后面加了一个'\0'，作为字符串结束标志。在执行printf函数时，每输出一个字符检查一次，看下一个字符是否为'\0'，遇'\0'就停止输出。

```
char c[]="I am happy";
```

```
或 char c[]="I am happy";
```

用一个字符串(注意字符串的两端是用双引号而不是单引号括起来的)作为字符数组的初值。

注意

- 数组c的长度不是10，而是11。因为字符串常量的最后由系统加上一个'\0'。

```
char c[]={'I',' ','a','m',' ','h','a','p','p','y','\0'}; ≠ char c[]={'I',' ','a','m',' ','h','a','p','p','y'};
```

```
char c[10]={"China"};
```

数组c的前5个元素为: 'C','h','i','n','a',第6个元素为'\0'，后4个元素也自动设定为空字符。

C	h	i	n	a	\0	\0	\0	\0	\0
---	---	---	---	---	----	----	----	----	----

8.1.5 数据结构1——数组与自定义数据类型

■ 字符串处理函数

- 输出字符串的函数
- 输入字符串的函数
- 字符串连接函数
- 字符串复制函数
- 字符串比较函数
- 测字符串长度的函数
- 转换为大小写的函数

注意

- 字符串处理函数属于库函数。库函数并非C语言本身的组成部分，而是C语言编译系统为方便用户使用而提供的公共函数。不同的编译系统提供的函数数量和函数名、函数功能都不尽相同，使用时要小心，必要时查一下库函数手册。
- 在使用字符串处理函数时，应当在程序文件的开头用`#include <string.h>`把string.h文件包含到本文件中。

8.1.5 数据结构1——数组与自定义数据类型

字符串处理函数

输出字符串的函数

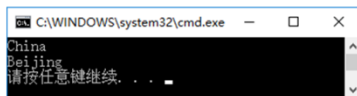
puts(字符数组)

作用：将一个字符串(以'\0'结束的字符序列)输出到终端。

用puts函数输出的字符串中可以包含转义字符。

在用puts输出时将字符串结束标志'\0'转换成'\n'，即输出完字符串后换行。

```
1. #include <stdio.h>
2. int main()
3. {
4.     char str[]={"China\nBeijing"};
5.     puts(str);
6.     return 0;
7. }
```



strcat(字符数组1, 字符数组2)

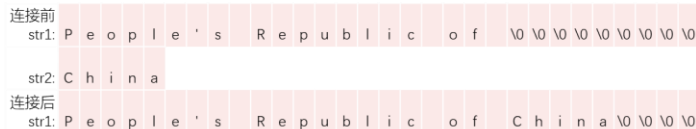
作用：把两个字符数组中的字符串连接起来，把字符串2接到字符串1的后面，结果放在字符数组1中，函数调用后得到一个函数值——字符数组1的地址。

字符数组1必须足够大，以便容纳连接后的新字符串。

连接前两个字符串的后面都有'\0'，连接时将字符串1后面的'\0'取消，只在新串最后保留'\0'。

```
char str1[30]={"People's Republic of "};
char str2[]={"China"};
printf("%s", strcat(str1, str2));
```

输出：People's Republic of China



输入字符串的函数

gets(字符数组)

作用：从终端输入一个字符串到字符数组，并且得到一个函数值。该函数值是字符数组的起始地址。

gets(str); //str是已定义的字符数组

如果从键盘输入：

Computer↵

将输入的字符串"Computer"送给字符数组str（请注意，送给数组的共有9个字符："Computer"外加一个'\0'），返回的函数值是字符数组str的第一个元素的地址。

注意

• 用puts和gets函数只能输出/输入一个字符串。



puts(str1, str2); 或 gets(str1, str2);

字符串复制函数

strcpy(字符数组1, 字符串2)

```
char str1[10], str2[]="China";
strcpy(str1, str2); 或 strcpy(str1, "China");
```

执行后，str1: C h i n a \0 \0 \0 \0 \0

- 作用：将字符串2复制到字符数组1中去。
- 字符数组1必须定义得足够大，以便容纳被复制的字符串2。字符数组1的长度不应小于字符串2的长度。
- "字符数组1"必须写成数组名形式，"字符串2"可以是字符数组名，也可以是一个字符串常量。
- 若在复制前未对字符数组1初始化或赋值，则其各字节中的内容无法预知，复制时将字符串2和其后的'\0'一起复制到字符数组1中，取代字符数组1中前面的字符，未被取代的字符保持原有内容。
- 不能用赋值语句将一个字符串常量或字符数组直接给一个字符数组。字符数组名是一个地址常量，它不能改变值，正如数值型数组名不能被赋值一样。
- 可以用strncpy函数将字符串str2中前面n个字符复制到字符数组str1中去。

strncpy(str1, str2, n);

将str2中最前面n个字符复制到str1中，取代str1中原有的最前面n个字符。但复制的字符个数n不应多于str1中原有的字符（不包括'\0'）。



str1="China"; str1=str2;

8.1.5 数据结构1——数组与自定义数据类型

字符串处理函数

有3个字符串,要求找出其中“最大”者

str[0]:	H	o	l	l	a	n	d	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0
str[1]:	C	h	i	n	a	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0
str[2]:	A	m	e	r	i	c	a	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0

```
1. #include<stdio.h>
2. #include<string.h>
3. int main()
4. {
5.     char str[3][20];    //定义二维字符数组
6.     char string[20];    //定义一维字符数组,作为交换字符串时的临时字符数组
7.     int i;
8.     for(i=0;i<3;i++) gets(str[i]); //读入3个字符串

9.     if(strcmp(str[0],str[1])>0)    //若str[0]大于str[1]
10.        strcpy(string,str[0]);    //把str[0]的字符串赋给字符数组string
11.     else
12.        strcpy(string,str[1]);    //把str[1]的字符串赋给字符数组string

13.     if(strcmp(str[2],string)>0)    //若str[2]大于string
14.        strcpy(string,str[2]);    //把str[2]的字符串赋给字符数组string

15.     printf("\nthe largest string is:\n%s\n",string); //输出string
16.     return 0;
17. }
```

字符串比较函数

strcmp(字符串1, 字符串2)

- 作用: 比较字符串1和字符串2。
- 字符串比较的规则是: 将两个字符串自左至右逐个字符相比(按ASCII码值大小比较), 直到出现不同的字符或遇到'\0'为止。
 - (1) 如全部字符相同, 则认为两个字符串相等;
 - (2) 若出现不相同的字符, 则以第1对不相同的字符的比较结果为准。
- 比较的结果由函数值带回。
 - (1) 如果字符串1与字符串2相同, 则函数值为0。
 - (2) 如果字符串1>字符串2, 则函数值为一个正整数。
 - (3) 如果字符串1<字符串2, 则函数值为一个负整数。
- 可以用strcmp函数比较字符串1和字符串2的前面n个字符。

strcmp(str1, str2, n);

测字符串长度的函数

strlen(字符数组)

作用: 测试字符串长度的函数。函数的值为字符串中的实际长度(不包括'\0'在内)。

```
1. #include <stdio.h>
2. #include <string.h>
3. int main()
4. {
5.     char str[10]="China";
6.     printf("%d,%d\n",strlen(str),sizeof(str));
7. }
```

转换为大小写的函数

strlwr(字符串)

作用: 将字符串中大写字母换成小写字母。

strupr(字符串)

作用: 将字符串中小写字母换成大写字母。

8.1.5 数据结构1——数组与自定义数据类型

■ 枚举类型

```
enum [枚举名]{枚举元素列表}
```

```
enum Weekday {sun, mon, tue, wed, thu, fri, sat};
```

如果一个变量只有几种可能的值，则可以定义为**枚举**(enumeration)**类型**，所谓“枚举”就是指把可能的值一一列举出来，变量的值只限于列举出来的值的范围内。声明枚举类型用enum开头。花括号中的sun, mon, ..., sat称为**枚举元素**或**枚举常量**。

```
enum Weekday weekday, weekend;
```

枚举类型

枚举变量

也可以不声明有名字的枚举类型，而直接定义枚举变量：

```
enum {sun, mon, tue, wed, thu, fri, sat} weekday, weekend;
```

- (1) C编译对枚举类型的枚举元素按常量处理，故称枚举常量。不要因为它们是标识符(有名字)而把它们看作变量，不能对它们赋值。
- (2) 每一个枚举元素都代表一个整数，C语言编译按定义时的顺序默认它们的值为0,1,2,3,4,5…。也可以在定义枚举类型时显式地指定枚举元素的数值。
- (3) 枚举元素可以用来作判断比较。枚举元素的比较规则是按其在初始化时指定的整数来进行比较的。

8.1.5 数据结构1——数组与自定义数据类型

■ 结构体

■ 一般规律:

- $\text{sizeof}(\text{对象}) = \text{sizeof}(\text{数据成员1}) + \text{sizeof}(\text{数据成员2}) + \dots + \text{sizeof}(\text{数据成员n})$
- $\text{address}(\text{数据成员i}) = \text{address}(\text{对象}) + \text{sizeof}(\text{数据成员1}) + \text{sizeof}(\text{数据成员2}) + \dots + \text{sizeof}(\text{数据成员i-1})$

■ 特殊情况:

- 空类: 类中没有任何数据成员
 - 若对象不占用内存空间, 就无法获得其地址
 - 空类的实际长度为1字节
- 内存对齐
 - 内存对齐可以加速数据成员的访问
 - 编译器默认将每个数据成员对齐至其大小的最近整数倍
 - 整个结构体的大小必须是结构体中占用内存空间最大的成员域大小的整数倍

■ 如何排布结构体成员域, 使得结构体所占内存最小?

```
1. struct Student
2. {
3.     int sno;
4.     char name[20];
5.     char sex;
6.     char class;
7.     enum Nation nation;
8.     short age;
9.     struct Date birthday;
10.};
```

`sizeof(struct Student) == 48`

```
1. struct Student2
2. {
3.     struct Date birthday;
4.     enum Nation nation;
5.     int sno;
6.     short age;
7.     char name[20];
8.     char sex;
9.     char class;
10.};
```

`sizeof(struct Student2) == 44`

方法: 把占用内存空间大的成员域放前面, 占用内存空间小的成员域放后面

C语言允许用户自己建立由不同类型数据组成的组合型的数据结构, 它称为**结构体 (structure)**。

在程序中建立一个结构体类型:

sno	name	sex	class	nation	age
2021123456	ZhangSan	F	T	2	18

```
enum Nation { bai, chaoxian, han, tujia, ... };
              0       1       2       3

struct Student
{
    int sno;           //学号为整型
    char name[20];     //姓名为字符串
    char sex;          //性别为字符型 (F: 女生, M: 男生)
    char class;        //班级为字符型 (T: 图灵, D: 金科)
    enum Nation nation; //民族为枚举类型
    short age;         //年龄为短整型
};                    //注意最后有一个分号
```

结构体类型的名字是由一个关键字**struct**和结构体名组合而成的。结构体名由用户指定, 又称“结构体标记”(structure tag)。

花括号内是该结构体所包括的子项, 称为结构体的成员(member)。对各成员都应进行类型声明, 即 **类型名 成员名;**

“成员列表”(member list)也称为“域表”(field list), 每一个成员是结构体中的一个域。成员名命名规则与变量名相同。

(1) 结构体类型并非只有一种, 而是可以设计出许多种结构体类型, 各自包含不同的成员。

(2) 成员可以属于另一个结构体类型。

sno	name	sex	class	nation	age	birthday		
						year	month	day

```
struct Date           //声明一个结构体类型 struct Date
{
    int year;          //年
    int month;         //月
    int day;           //日
};
```

```
struct Student        //声明一个结构体类型 struct Student
{
    int sno;
    char name[20];
    char sex;
    char class;
    enum Nation nation;
    short age;
    struct Date birthday; //成员birthday属于struct Date类型
};
```


8.1.5 数据结构1——数组与自定义数据类型

■ 共用体

```
12. struct Student
13. {
14.     int sno;
15.     char name[20];
16.     char sex;
17.     char class;
18.     enum Nation nation;
19.     int age;
20.     struct Date birthday;
21.     char category;
22.     union Extra extra;
23. }
```

```
1. #include <stdio.h>
2. enum Nation {bai, chaoxian, han, tujia, yao};
3. struct Date {int year; int month; int day;};
4. union Extra {
5.     float score;
6.     char subject;
7. };

8. int main() {
9.     printf("%d %d\n", sizeof(struct Extra),
10.            sizeof(struct Student));
11. }
```

4 56

使几个不同类型的变量共享同一段内存的结构，称为“共用体”类型的结构。

union 共用体名 {
成员表列
} 变量表列;

1000地址

短整型变量i

字符型变量ch

实型变量f

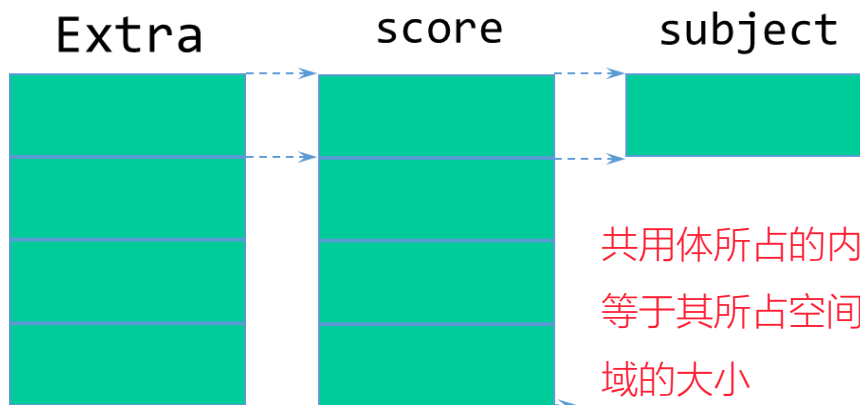
```
union Data //表示不同类型的变量i,ch,f可以存放到同一段存储单元中
{
    int i;
    char ch;
    float f;
} a,b,c; //在声明类型同时定义变量
```

```
union Data //声明共用体类型
{
    int i;
    char ch;
    float f;
};
union Data a,b,c; //用共用体类型定义变量
```

```
union //没有定义共用体类型名
{
    int i;
    char ch;
    float f;
} a,b,c;
```

“共用体”与“结构体”的定义形式相似。但它们的含义是不同的。

- 结构体变量所占内存长度是各成员占的内存长度之和。每个成员分别占有其自己的内存单元。
- 而共用体变量所占的内存长度等于最长的成员的长度。几个成员共用一个内存区。



共用体所占的内存空间大小
等于其所占空间最大的成员
域的大小

8.1.5 数据结构1——数组与自定义数据类型

■ typedef

1. 简单用一个新的类型名代替原有类型名

```
typedef int Integer; //指定用Integer为类型名, 作用与int相同
typedef float Real; //指定用Real为类型名, 作用与float相同
```

2. 命名一个简单的类型名代替复杂的类型表示方法

① 命名一个新类型名代表结构体类型

② 命名一个新类型名代表数组类型

③ 命名一个新类型名代表指针类型

④ 命名一个新类型名代表指向函数的指针类型

```
typedef struct
{
    int month;
    int day;
    int year;
}Date;           //声明了一个新类型名Date, 代表结构体类型
Date birthday;   //定义结构体类型变量birthday, 不要写成struct Date birthday;
Date*p;          //定义结构体指针变量p, 指向此结构体类型数据 ①

typedef int Num[100]; //声明Num为整型数组类型名
Num a;             //定义a为整型数组名, 它有100个元素 ②

typedef char*String; //声明String为字符指针类型
String p,s[10];      //定义p为字符指针变量, s为字符指针数组 ③

ypedef int (*Pointer)(); //声明Pointer为指向函数的指针类型, 该函数返回整型值
Pointer p1,p2;        //p1,p2为Pointer类型的指针变量 ④
```

8.1.5 数据结构1——数组与自定义数据类型

■ 枚举/结构体/共用体综合应用实例

■ 定义Student数据结构

- 学号: int sno
- 姓名: char name[20]
- 年龄: int age
- 性别: char sex ('M': 男性, 'F': 女性)
- 班级: char class ('T': 图灵班, 'D': 金科班)
- 民族: enum Nation { bai, chaoxian, han, tujia, yao, ... } nation
- 生日: struct Date { int year; int month; int day; } birthday
- 类别: char category ('N': 统考生; 'B': 保送生)
- 其他: union Extra { float score; char subject; } extra
 - score: 高考成绩
 - subject: 获奖的竞赛学科 ('M': 数学, 'P': 物理, 'I': 信息学, 'C': 化学, 'B': 生物)

```
typedef struct
{
    int sno;
    char name[20];
    char sex;
    char class;
    enum Nation nation;
    int age;
    struct Date birthday;
    char category;
    union Extra extra;
} Student;
```

■ 定义学生结构体数组并赋初值

```
Student students[] = {
    {2021123456, "ZhangSan", 'F', 'T', han, 18, {2003, 10, 28}, 'B', 'I'},
    {2021654321, "LiSi", 'M', 'D', yao, 19, {2002, 11, 22}, 'N', 680}
};
```

■ 查看今天的日期

```
#include <time.h>
struct tm {
    int tm_sec; //代表目前秒数, 正常范围为0-59, 但允许至61秒
    int tm_min; //代表目前分数, 范围0-59
    int tm_hour; //从午夜算起的时数, 范围为0-23
    int tm_mday; //目前月份的日数, 范围01-31
    int tm_mon; //代表目前月份, 从一月算起, 范围从0-11
    int tm_year; //从1900 年算起至今的年数
    int tm_wday; //一星期的日数, 从星期一算起, 范围为0-6
    int tm_yday; //从今年1 月1 日算起至今的天数, 范围为0-365
    int tm_isdst; //日光节约时间的旗标
};
time_t time(time_t * ); //返回从1970年1月1日 (MFC是1899年12月31日) 0时0分0秒, 到现在的秒数
struct tm * localtime(const time_t * timer); //将日历时间转为本地时间
```

8.1.5 数据结构1——数组与自定义数据类型

■ 枚举/结构体/共用体综合应用实例

```
1. #include <stdio.h>
2. #include <time.h>

3. typedef enum {
4.     bai, chaoxian, han, tujia, yao
5. } Nation;

6. typedef struct {
7.     int year; int month; int day;
8. } Date;

9. typedef union {
10.     float score; char subject;
11. } Extra;

12. typedef struct
13. {
14.     int sno;
15.     char name[20];
16.     char sex;
17.     char class;
18.     Nation nation;
19.     int age;
20.     Date birthday;
21.     char category;
22.     Extra extra;
23. } Student;
```

```
25. int main() {
26.     time_t rawtime;
27.     struct tm *target_time;
28.     int this_year, this_month, this_day, i;

29.     Student students[] = {
30.         {2021123456, "ZhangSan", 'F', 'T', han, 18,
31.          {2003, 10, 28}, 'B', 'I'},
32.         {2021654321, "LiSi", 'M', 'D', yao, 19,
33.          {2002, 11, 22}, 'N', 680}
34.     };

35.     time(&rawtime);
36.     target_time = localtime(&rawtime);
37.     this_year = target_time->tm_year + 1900;
38.     this_month = target_time->tm_mon + 1;
39.     this_day = target_time->tm_mday;
40.     printf("Today is: %d-%d-%d\n",
41.            this_year, this_month, this_day);

42.     for (i=0; i<sizeof(students)/sizeof(struct Student); i++)
43.         if (students[i].birthday.month == this_month &&
44.             students[i].birthday.day == this_day) {
45.             printf("\t%s(%d)'s %d birthdy.\n",
46.                    students[i].name, students[i].sno, students[i].age);
47.         }
```

8.1.5 数据结构1——数组与自定义数据类型

■ 位域

■ 位域的声明形式

- 数据类型说明符 成员名：位数;

■ 位域的作用

- 通过“打包”，使类的不同成员共享相同的字节，从而节省存储空间。

■ 注意事项

- 节省空间，但可能增加时间开销
- 具体的打包方式，因编译器而异；
- 只有char、int和枚举类型的成员，允许定义为位域；
- 位域指定的位数必须小于等于该类型所占的位数；
- 由于多个位域可能存储在同一个存储单元中，所以不可以对位域执行取地址操作，否则会出现编译错误；
- 引用位段时，系统自动将它转换成整型；
- 位段可以用%d、%u、%x、%o和%c的格式输出

```
1. struct Student3
2. {
3.     struct Date birthday;
4.     union Extra extra;
5.     enum Nation nation;
6.     int sno;
7.     short age;
8.     char name[20];
9.     char sex;
10.    char class;
11.    char category;
12.};
```

sizeof(struct Student3) == 52

```
1. struct Student4
2. {
3.     struct Date birthday;
4.     union Extra extra;
5.     enum Nation nation;
6.     int sno;
7.     short age;
8.     char name[20];
9.     char sex : 1;
10.    char class : 1;
11.    char category : 1;
12.};
```

sizeof(struct Student4) == 48

8.1.5 数据结构1——数组与自定义数据类型

■ 位域

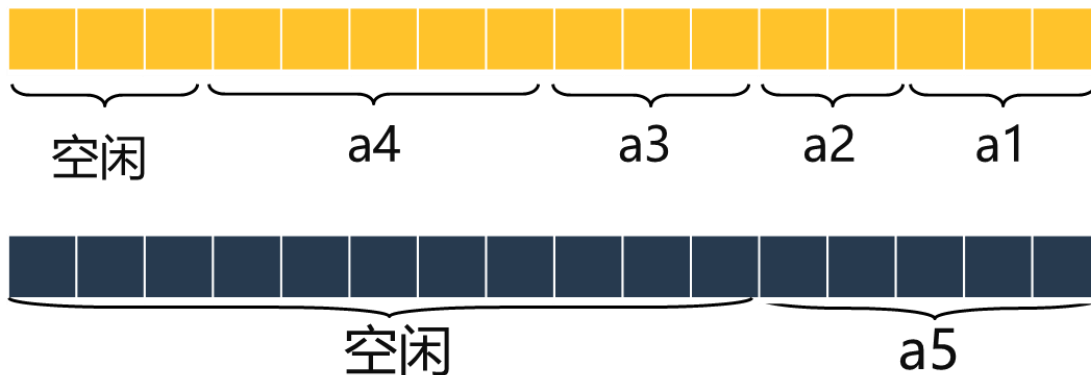
■ 位域在内存中的存储

```
struct sample {  
    int no;  
    char c1:1;  
    char c2:2;  
    char c3:1;  
    char c4:1;  
    short a1:3;  
    short a2:2;  
    short a3:3;  
    short a4:5;  
    short a5:5;  
};
```

相同类型的位段可以在同一块空间中



一个位段必须存储在一个存储单元中，不能跨两个存储单元。



8.1.5 数据结构1——数组与自定义数据类型

■ 位运算

运算符	&		^	~	<<	>>
含义	按位与	按位或	按位异或	取反	左移	右移

■ 注意事项

- 参加位运算的数据类型只能是整型或字符型
- 取反是一元运算，其他都是二元运算
- 负数按补码形式参加按位与运算

8.1.5 数据结构1——数组与自定义数据类型

■ 位运算

■ 按位与 (&)

运算规则

$0 \& 0 = 0$
 $1 \& 0 = 0$
 $0 \& 1 = 0$
 $1 \& 1 = 1$

用途

- 用 $0 \& x = 0$ ，将一个数值中的某些位清0
- 用 $1 \& x = x$ ，检验一个数值中某些位的值

例, $4 \& 6 = 4$

00000000	00000000	00000000	00000100
&	00000000	00000000	00000110

00000000	00000000	00000000	00000100

■ 按位或 (|)

运算规则

$0 | 0 = 0$
 $1 | 0 = 1$
 $0 | 1 = 1$
 $1 | 1 = 1$

用途

- 利用按位或运算将某些位置1

例, $4 | 6 = 6$

00000000	00000000	00000000	00000100
	00000000	00000000	00000110

00000000	00000000	00000000	00000110

■ 按位异或 (^)

运算规则

$0 \wedge 0 = 0$
 $1 \wedge 0 = 1$
 $0 \wedge 1 = 1$
 $1 \wedge 1 = 0$

用途

- 利用按位异或将某些位取反
- 利用按位异或交换两个变量值

$4 \wedge 6 = 2$

00000000	00000000	00000000	00000100
^	00000000	00000000	00000110

00000000	00000000	00000000	00000010

■ 取反 (~)

运算规则

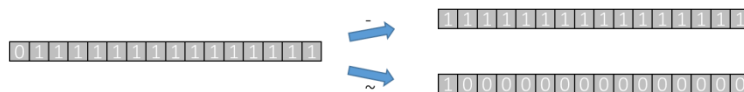
$\sim 0 = 1$
 $\sim 1 = 0$

用途

- 将某数的每一位取反

■ ~和-运算的区别

- ~操作是把每一位都取反，即0变1，1变0
- -运算是把正数变成负数，负数变成正数
- 例：短整型变量x的值为32767，则-x的值为-32767，而~x的值为-32768



8.1.5 数据结构1——数组与自定义数据类型

■ 位运算

■ 左移 (<<)

将一个数的二进制表示中的每个位向左移动若干位。

例如： $a = a << 2$ ；表示将a的二进制表示中的每一位向左移动两位，右边补0，左移后溢出的高位被舍弃。

- 若a的值为15，即二进制的00001111，执行这个操作后，a的值为60，即二进制的00111100。
- 若a的值为-127，即二进制的10000001，执行这个操作后，a的值为4，即二进制的00000100。

- ▶ 向左移动1位表示乘2。
- ▶ 由于移位操作比乘法操作执行速度快，所以有经验的程序员经常会用左移操作代替乘2的操作。
- ▶ 15向左移动2位表示 $15 * 2 * 2 = 60$ ，-1左移2位变成了-4。

■ 右移 (>>)

将一个数的二进制表示中的每个位向右移动若干位。

例如： $a = a >> 2$ ；表示将a的二进制表示中的每一位向右移动两位。

左边的填充：

- 无符号整数和正整数，左边补0。
- 负数，取决于编译环境。若系统采用逻辑右移，则填充0。若系统采用的是算术右移，则填充1。

- ▶ 向右移动1位表示除2
- ▶ 由于移位操作比乘法操作执行速度快，所以有经验的程序员经常会用左移操作代替乘2的操作。

例：

15向右移动2位表示 $15 / 2 / 2 = 3$ ，即
 $00001111 >> 2 = 00000011$



8.1.6 算法2——查找与排序

■ 查找

■ 顺序查找

- 适用范围：一般针对无序表的查找
- 方法：从数组的第一个元素开始，将被查找数与数组元素进行比较，直到找到或确定不存在为止

■ 折半查找

- 适用范围：有序表
- 方法：
 - 设置变量low和high分别指向查找数据段的起止位置，用mid指向中间位置，
 - 将被查找数与mid位置指向的数进行比较
 - 若被查找数大于mid位置指向的数，则将low与mid之间的数折掉， low定位于mid+1位置；
 - 若被查找数小于mid位置指向的数，则将mid 与high之间的数折掉， high定位于mid-1位置；
 - 继续求mid位置，并比较被查找数与mid位置指向的数，直到找到或确定不存在为止

8.1.6 算法2——查找与排序

■ 排序

选择 排序

从所有的数中找出最小的一个，将其放在最前面；接着在余下的数中找出最小的一个，将其放在第二位，依次类推，数列由前往后逐渐成型。

冒泡 排序

对相邻两个数进行比较，将较小的调到前面，两两比较一轮之后，最大的一个数被放置在最后面；接着从头开始重复执行以上操作，次大的数被放置在倒数第二位，依次类推，数列由后往前逐渐成型。

8.1.6 算法2——查找与排序

■ 查找和排序应用举例

■ 1. 使用选择排序将通讯录按学号信息排序

```
1. #include <stdio.h>

2. typedef enum { bai, chaoxian, han, tujia, yao } Nation;
3. typedef struct { int year; int month; int day; } Date;
4. typedef struct {
5.     int sno;
6.     char name[20];
7.     char sex;
8.     char class;
9.     Nation nation;
10.    int age;
11.    Date birthday;
12.} Student;

13.int main(void)
14.{
15.    char nations[][10] = {"bai", "chaoxian", "han", "tujia", "yao"};
16.    Student students[] =
17.    {
18.        {2021123456, "ZhangSan", 'F', 'T', han, 18, {2003, 8, 1}},
19.        {2021200834, "LiSi", 'M', 'D', yao, 19, {2002, 6, 25}},
20.    };
```

8.1.6 算法2——查找与排序

■ 查找和排序应用举例

■ 1. 使用选择排序将通讯录按学号信息排序

```
21. int i, j, min;
22. Student temp;

23. for (i = 0; i < sizeof(students) / sizeof(Student) - 1; i++) {
24.     min = i;
25.     for (j = i + 1; j < sizeof(students) / sizeof(Student); j++)
26.         if (students[min].sno > students[j].sno)
27.             min = j;
28.     if (min != i) {
29.         temp = students[i];
30.         students[i] = students[min];
31.         students[min] = temp;
32.     }
33. }

34. for (i = 0; i < sizeof(students) / sizeof(Student); i++)
35.     printf("%d %s %c %c %s %d %d-%d-%d\n",
36.         students[i].sno, students[i].name,
37.         students[i].sex, students[i].class,
38.         nations[students[i].nation], students[i].age,
39.         students[i].birthday.year, students[i].birthday.month, students[i].birthday.day);
40. }
```

8.1.6 算法2——查找与排序

■ 查找和排序应用举例

■ 2. 使用冒泡排序将通讯录按姓名信息排序

```
1.  int i, j;
2.  Student temp;

3.  for (i = 0; i < sizeof(students) / sizeof(Student) - 1; i++)
4.  {
5.      for (j = 0; j < sizeof(students) / sizeof(Student) - i - 1; j++)
6.          if (strcmp(students[j].name, students[j + 1].name) > 0)
7.          {
8.              temp = students[j];
9.              students[j] = students[j + 1];
10.             students[j + 1] = temp;
11.         }
12. }
```

8.1.6 算法2——查找与排序

■ 查找和排序应用举例

■ 3. 多字段排序——按生日排序

```
1.  int i, j;
2.  int is_big, encode1, encode2;
3.  Student temp;

4.  for (i = 0; i < sizeof(students) / sizeof(Student) - 1; i++)
5.  {
6.      for (j = 0; j < sizeof(students) / sizeof(Student) - i - 1; j++)
7.      {
8.          encode1 = students[j].birthday.year * 10000 + students[j].birthday.month * 100 +
9.          students[j].birthday.day;
10.         encode2 = students[j+1].birthday.year * 10000 + students[j+1].birthday.month * 100 +
11.         students[j + 1].birthday.day;

12.         if (encode1 < encode2)
13.         {
14.             temp = students[j];
15.             students[j] = students[j + 1];
16.             students[j + 1] = temp;
17.         }
18.     }
19. }
```

适用场景：各字段
类型相同且取值范
围固定

巧用自定义编码
进行多字段排序

8.1.6 算法2——查找与排序

■ 查找和排序应用举例

■ 3. 多字段排序——按生日排序

```
1.  int i, j;  
2.  int is_big, diff_year, diff_month, diff_day;  
3.  Student temp;  
  
4.  for (i = 0; i < sizeof(students) / sizeof(Student) - 1; i++)  
5.  {  
6.      for (j = 0; j < sizeof(students) / sizeof(Student) - i - 1; j++)  
7.      {  
8.          diff_year = students[j].birthday.year - students[j + 1].birthday.year;  
9.          diff_month = students[j].birthday.month - students[j + 1].birthday.month;  
10.         diff_day = students[j].birthday.day - students[j + 1].birthday.day;  
  
11.         is_big = (diff_year != 0 ? diff_year < 0 :  
12.             (diff_month != 0 ? diff_month < 0 : diff_day < 0));  
  
13.         if (is_big)  
14.         {  
15.             temp = students[j];  
16.             students[j] = students[j + 1];  
17.             students[j + 1] = temp;  
18.         }  
19.     }  
20. }
```

相比于自定义编码
的方式更为通用

巧用逻辑表达式
进行多字段排序

8.1.6 算法2——查找与排序

■ 查找和排序应用举例

■ 4. 顺序查找学号

```
1.  int i, sno, found = -1;
2.  scanf("%d", &sno);
3.  for (i = 0; i < sizeof(students) / sizeof(Student) - 1; i++)
4.  {
5.      if (students[i].sno == sno) {
6.          found = i;
7.          break;
8.      }
9.  }
10. if (found >= 0) {
11.     printf("%d %s %c %c %s %d %d-%d-%d\n",
12.         students[found].sno, students[found].name,
13.         students[found].sex, students[found].class,
14.         nations[students[found].nation], students[found].age,
15.         students[found].birthday.year,
16.         students[found].birthday.month,
17.         students[found].birthday.day);
18. } else printf("not found\n");
```


8.1.6 算法2——查找与排序

■ 查找和排序应用举例

■ 5. 折半查找学号

```
1.  int i, sno, found = -1;
2.  int low = 0, high = sizeof(students) / sizeof(Student) - 1;
3.  int mid;

4.  scanf("%d", &sno);

5.  while (low <= high)
6.  {
7.      mid = (low + high) / 2;
8.      if (sno > students[mid].sno)
9.          low = mid + 1;
10.     else if (sno < students[mid].sno)
11.         high = mid - 1;
12.     else
13.     {
14.         found = mid;
15.         break;
16.     }
17. }
```

8.2 基本功能代码

- 数据输入
- 循环分支
- 数位分离
- 进制转换
- 数组操作
- 字符串操作
- 排序
- 素数

8.2.1 数据输入

■ 读入n个数据，存放在数组中

```
1.  int n, i;
2.  int arr[100];
3.  scanf("%d", &n);
4.  for (i = 0; i < n; i++)
5.      scanf("%d", &arr[i]);
```

■ 读入若干数据，-1为结束标志

```
1.  int n = 0, tmp;
2.  int arr[100];
3.  do {
4.      scanf("%d", &tmp);
5.      arr[n++] = tmp;
6.  } while (tmp != -1);
```

■ 字符和数字混合读

■ R□255□G□255□B□255

```
1.  char ch1, ch2, ch3;
2.  int num1, num2, num3;
3.  scanf("%c%d□%c%d□%c%d", &ch1, &num1, &ch2, &num2, &ch3, &num3);
4.  printf("%c%d%c%d%c%d", ch1, num1, ch2, num2, ch3, num3);
```

■ R255G144B255

```
1.  char ch1, ch2, ch3;
2.  int num1, num2, num3;
3.  scanf("%c%3d%c%3d%c%3d", &ch1, &num1, &ch2, &num2, &ch3, &num3);
4.  printf("%c%d%c%d%c%d", ch1, num1, ch2, num2, ch3, num3);
```

■ R255G14B255

```
1.  char ch1, ch2, ch3;
2.  int num1, num2, num3;
3.  scanf("%c%d%c%d%c%d", &ch1, &num1, &ch2, &num2, &ch3, &num3);
4.  printf("%c%d%c%d%c%d", ch1, num1, ch2, num2, ch3, num3);
```

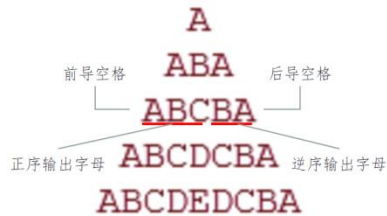
8.2.2 循环分支

■ 打印图形

```

1.  #include <stdio.h>
2.  void main()
3.  {
4.      int n, i, j;
5.      scanf("%d", &n);
6.      for (i = 1; i <= n; i++)
7.      {
8.          for (j = 1; j <= n - i; j++)
9.              printf(" ");           //输出前导空格
10.         for (j = 1; j <= i; j++)
11.             printf("%c", 'A'+j-1); //正序输出字母
12.         for (j = i; j > 1; j--)
13.             printf("%c", 'A'+j);   //逆序输出字母
14.         for (j = 1; j <= n - i; j++)
15.             printf(" ");           //输出后导空格
16.         printf("\n");              //换行
17.     }
18. }

```



```

1.  void main()
2.  {
3.      int score;
4.      float gpa;
5.      scanf("%d", &score);
6.      if (score >= 90) gpa = 4.0;
7.      else if (score >= 86) gpa = 3.7;
8.      else if (score >= 83) gpa = 3.3;
9.      else if (score >= 80) gpa = 3.0;
10.     else if (score >= 76) gpa = 2.7;
11.     else if (score >= 73) gpa = 2.3;
12.     else if (score >= 70) gpa = 2.0;
13.     else if (score >= 66) gpa = 1.7;
14.     else if (score >= 63) gpa = 1.3;
15.     else if (score >= 60) gpa = 1.0;
16.     else gpa = 0;
17.     if (gpa != 0) printf("%.1f\n", gpa);
18.     else printf("0\n");
19. }

```

```

1.  a = score / 10;
2.  b = score % 10;
3.
4.  switch (a) {
5.      case 9: case 10: gpa = 4; break;
6.      case 8: gpa = 3; break;
7.      case 7: gpa = 2; break;
8.      case 6: gpa = 1; break;
9.      default: gpa = 0;
10. }
11.
12. if (gpa > 0 and gpa < 4) {
13.     switch (b) {
14.         case 6: case 7: case 8: case 9: gpa += 0.4;
15.         case 3: case 4: case 5: gpa += 0.3;
16.     }
17. }

```

8.2.3 数位分离

■ 整数转换为数字字符串

```
1.  #include <stdio.h>
2.  void main() {
3.      int x = 123456789;
4.      char str[100] = {0}, tmp;
5.      int idx = 0, digit, i;
6.      //L7-11: 解码每一位数码
7.      while (x > 0) {
8.          digit = x % 10;
9.          x = x / 10;
10.         str[idx++] = digit + '0';
11.     }
12.     printf("%s\n", str);
13.     //L14-18: 字符串翻转
14.     for (i = 0; i < idx / 2; i++) {
15.         tmp = str[i];
16.         str[i] = str[idx - i - 1];
17.         str[idx - i - 1] = tmp;
18.     }
19.     printf("%s\n", str);
20. }
```

987654321
123456789

■ 数字字符串转为整数

```
1.  #include <stdio.h>
2.  void main()
3.  {
4.      char str[] = "123456789";
5.      int x = 0;
6.      int weight = 1, base = 10;
7.      int i;
8.      //L9-13: 数码按位权进行编码
9.      for (i = strlen(str) - 1; i >= 0; i--)
10.     {
11.         x = x + (str[i] - '0') * weight;
12.         weight = weight * base;
13.     }
14.     printf("%d\n", x);
15. }
```

123456789

8.2.4 进制转换

■ 10进制转16进制

```
1.  #include <stdio.h>
2.  void main()
3.  {
4.      long long input;
5.      char symbol[16] = {'0', '1', '2', '3',
6.                          '4', '5', '6', '7',
7.                          '8', '9', 'A', 'B',
8.                          'C', 'D', 'E', 'F'};
9.      char output[100] = "";
10.     int idx = 0, base = 16, tmp, i;
11.     scanf("%lld", &input);
12.     //L13-18: 除基数取余数
13.     while (input > 0)
14.     {
15.         tmp = input % base;
16.         output[idx++] = symbol[tmp];
17.         input = input / base;
18.     }
19.     //L20-25: 字符串翻转
20.     for (i = 0; i < idx / 2; i++)
21.     {
22.         tmp = output[i];
23.         output[i] = output[idx - i - 1];
24.         output[idx - i - 1] = tmp;
25.     }
26.     printf("%s\n", output);
27. }
```

■ 16进制转10进制

```
1.  #include <stdio.h>
2.  void main() {
3.      long long output = 0;
4.      char input[100];
5.      int len, i, tmp, weight = 1, base = 16;
6.
7.      scanf("%s", input);
8.      len = strlen(input);
9.      //L9-18: 按位权展开求和
10.     for (i = len - 1; i >= 0; i--) {
11.         if (input[i] >= '0' && input[i] <= '9')
12.             tmp = input[i] - '0';
13.         if (input[i] >= 'A' && input[i] <= 'F')
14.             tmp = input[i] - 'A' + 10;
15.         if (input[i] >= 'a' && input[i] <= 'f')
16.             tmp = input[i] - 'a' + 10;
17.         output = output + tmp * weight;
18.         weight = weight * base;
19.     }
20.     printf("%lld", output);
21. }
```

8.2.5 数组操作

■ 找最大、最小、求和、平均数 ■ 统计数组中的某些项数

```
1.  #include <stdio.h>
2.  void main()
3.  {
4.      int arr[10] = {1, 2, 3, 4, 4, 5, 2, 6, 7, 9};
5.      int min = 10, max = 0, sum = 0; //赋初值, 切记
6.      float avg;
7.      int i;
8.      for (i = 0; i < 10; i++)
9.      {
10.         if (arr[i] > max)
11.             max = arr[i];
12.         if (arr[i] < min)
13.             min = arr[i];
14.         sum = sum + arr[i];
15.     }
16.     avg = (float)(sum) / 10;
17.     //错误写法: avg = (float)(sum / 10);
18.     printf("max: %d min: %d sum: %d avg: %f",
19.         max, min, sum, avg);
20. }
```

```
1.  #include <stdio.h>
2.  void main() {
3.      char str[100];
4.      int len, i;
5.      int a = 0, e = 0, i = 0, o = 0, u = 0;
6.
7.      gets(str);
8.      len = strlen(str);
9.      for (i = 0; i < len; i++) {
10.         if (str[i] == 'a' || str[i] == 'A') a++;
11.         if (str[i] == 'e' || str[i] == 'E') e++;
12.         if (str[i] == 'i' || str[i] == 'I') i++;
13.         if (str[i] == 'o' || str[i] == 'O') o++;
14.         if (str[i] == 'u' || str[i] == 'U') u++;
15.     }
16. }
```

8.2.5 数组操作

■ 矩阵转置

```
1.  #include <stdio.h>
2.  #define MAX 1000

3.  int a[MAX][MAX], b[MAX][MAX]; //大容量数组在全局定义

4.  void main()
5.  {
6.      int n, m;
7.      int i, j;
8.
9.      scanf("%d %d", &n, &m);
10.
11.      for (i = 0; i < n; i++)
12.          for (j = 0; j < m; j++)
13.              scanf("%d", &a[i][j]);
14.
15.      for (i = 0; i < n; i++)
16.          for (j = 0; j < m; j++)
17.              b[j][i] = a[i][j];
18.
19.      for (i = 0; i < m; i++)
20.      {
21.          for (j = 0; j < n; j++)
22.              printf("%d ", b[i][j]);
23.          printf("\n");
24.      }
```

■ 矩阵对角线各数的和

```
1.  #include <stdio.h>
2.  #define MAX 1000

3.  int x[MAX][MAX]; //大容量数组在全局定义

4.  void main() {
5.      int n;
6.      int i, j;
7.      int left_diagonal = 0, right_diagonal = 0;
8.
9.      scanf("%d", &n);
10.
11.      for (i = 0; i < n; i++)
12.          for (j = 0; j < n; j++)
13.              scanf("%d", &x[i][j]);
14.
15.      for (i = 0; i < n; i++)
16.      {
17.          left_diagonal += x[i][i];
18.          right_diagonal += x[i][n - i - 1];
19.      }
20.
21.      printf("left_diagonal: %d right_diagonal: %d\n",
22.             left_diagonal, right_diagonal);
23. }
```


8.2.6 字符串操作

■ 字符串比较

```
1.  #include <stdio.h>
2.  #define MAX_LEN 100

3.  void main() {
4.      int i, result = 0;
5.      char s1[MAX_LEN], s2[MAX_LEN];

6.      scanf("%s %s", s1, s2);

7.      //遇到s1或s2的结束符就退出循环
8.      for (i = 0; s1[i] != '\0' && s2[i] != '\0'; i++)
9.      {
10.         //什么情况下继续比较下一个?
11.         if ( s1[i] == s2[i] ) continue;
12.         else {
13.             result = s1[i] - s2[i];
14.             //已比较出结果, 无需继续循环
15.             break ;
16.         }
17.     }

18.     printf("%d\n", result);
19. }
```

■ 字符串复制

```
1.  #include <stdio.h>
2.  #define MAX_LEN 100

3.  void main() {
4.      int i;
5.      char s1[MAX_LEN], s2[MAX_LEN];

6.      scanf("%s", s1);

7.      i = 0;
8.      do {
9.          s2[i] = s1[i] ;    //单个字符复制
10.         i++ ;    //循环计数器累增
11.     } while ( s1[i] != '\0' );
12.     //只要没遇到s1的结束符, 就继续循环

13.     printf("%s", s2);
14. }
```

■ 字符串拼接

```
1.  #include <stdio.h>
2.  #define MAX_LEN 100

3.  void main() {
4.      int i, j;
5.      char s1[MAX_LEN], s2[MAX_LEN];
6.      scanf("%s %s", s1, s2);

7.      i = 0;
8.      j = 0;

9.      //只要未遇到s1的结束符, 就继续循环
10.     while ( s1[i] != '\0' ) i++;
11.
12.     //只要未遇到s2的结束符, 就继续循环
13.     while ( s2[j] != '\0' )
14.     {
15.         s1[i+j] = s2[j] ; //处理s2的每个字符
16.         j++;
17.     }

18.     printf("%s", s1);
19. }
```

8.2.6 字符串操作

■ 子串替换

- 编写一个字符串处理函数，其功能是在字符串A中查找字符串B，将A中出现的第一个B用字符串C替换，函数返回1；如果A中不存在字符串B，则将字符串C连接到字符串A的后面，函数返回0
- 函数的定义如下：

int replaceStr(char A[], const char B[], const char C[]) { }

```
1. #include <stdio.h>
2. #include <string.h>

3. int replaceStr(char A[], char B[], char C[]);

4. void main() {
5.     char str1[N], str2[N], str3[N];
6.     int nRes;
7.     gets(str1);
8.     gets(str2);
9.     gets(str3);
10.    nRes = replaceStr(str1, str2, str3);
11.    printf("%d\n", nRes);
12.    printf("%s\n", str1);
13.}
```

8.2.6 字符串操作

■ 子串替换

```
1.  int replaceStr(char A[], char B[], char C[]) {
2.      int len_A = strlen(A);
3.      int len_B = strlen(B);
4.      int len_C = strlen(C);
5.      int exist = 0, i, j, k;

6.      for (i = 0; i < len_A - len_B; i++) { //从A的第i个位置开始查子串
7.          for (j = 0; j < len_B; j++)
8.              if (A[i+j] != B[j]) break;          //这个break退出哪个循环?
9.          if (j < len_B) continue;                //这个continue会有何影响?
10.         exist = 1;
11.         if (len_B == len_C) {                    //如果被替换的子串B与替代串C长度相等
12.             for (k = 0; k < len_C; k++) A[i+k] = C[k];
13.         } else if (len_B >= len_C) {              //如果被替换的子串B长于替换串C
14.             for (k = 0; k < len_C; k++) A[i+k] = C[k];
15.             for (k = i+len_C; k <= len_A-(len_B-len_C); k++) A[k] = A[k+(len_B-len_C)];
16.         } else {                                  //如果被替换的子串B短于替换串C
17.             for (k = len_A; k >= i + len_B; k--) A[k+(len_C-len_B)] = A[k];
18.             for (k = 0; k < len_C; k++) A[i+k] = C[k];
19.         }
20.         break;                                    //这个break退出哪个循环?
21.     }
22.
23.     if (!exist) strcat(A, C);
24.     return exist;
25. }
```

8.2.7 排序

■ 简单冒泡排序（带优化）

```
1. void main()
2. {
3.     int arr[10], i, j, temp;
4.     int swap;                                //标记每趟扫描是否有发生交换：0未发生，1发生

5.     printf("Please input 10 numbers:\n");
6.     for ( i = 0; i < 10; i++) scanf("%d", &arr[i]);

7.     for (i = 0; i < 10 - 1; i++) {
8.         swap = 0;                            //每趟扫描开始时，设置swap为0
9.         for ( j = 0; j < 10 - i - 1; j++)
10.            if(arr[j] > arr[j+1]) {
11.                temp = arr[j];
12.                arr[j] = arr[j+1];
13.                arr[j+1] = temp;
14.                swap = 1;                    //如果发生了交换，设置swap为1
15.            }
16.         if (swap == 0) break;                //如果某趟循环没有发生交换，则提前结束排序
17.     }

18.     printf("The sorted numbers:\n");
19.     for ( i = 0; i < 10; i++) printf("%4d", arr[i]);
20. }
```

8.2.7 排序

■ 针对字符串进行的排序（使用选择排序）

```
1. #define N 100
2. #define LEN 101
3. char str[N][LEN];

4. void main() {
5.     int n, i, j, min;
6.     char temp[LEN];           //交换字符串时使用的临时变量，是一个字符数组

7.     scanf("%d", &n);
8.     for (i = 0; i < n; i++) scanf("%s", str[i]);

9.     for (i = 0; i < n - 1; i++) {
10.         min = i;
11.         for (j = i + 1; j < n; j++)
12.             if (strcmp(str[min], str[j]) > 0)    //字符串比较使用strcmp函数，返回两个字符串第一个不相同字符的ASCII码差值
13.                 min = j;
14.         if (min != i) {
15.             strcpy(temp, str[i]);
16.             strcpy(str[i], str[min]);
17.             strcpy(str[min], temp);
18.         }
19.     }

20.     for (i = 0; i < n; i++) printf("%s\n", str[i]);
21. }
```

8.2.7 排序

■ 自定义规则排序（方法一：自定义编码）

```
1. #include <stdio.h>
2. #define N 100

3. typedef struct { unsigned char A; unsigned char B; unsigned char C; } Data; //A,B,C类型相同, 取值范围均为[0,255]
4. Data data[N];

5. void main() {
6.     int n, i, j;
7.     int encode1, encode2;
8.     Data temp;

9.     scanf("%d", &n);
10.    for (i = 0; i < n; i++) scanf("%d %d %d", &data[i].A, &data[i].B, &data[i].C);

11.    for (i = 0; i < n; i++)
12.        for (j = 0; j < n - i - 1; j++) {
13.            encode1 = data[j].A * 256 * 256 + data[j].B * 256 + data[j].C; //自定义编码
14.            encode2 = data[j+1].A * 256 * 256 + data[j+1].B * 256 + data[j+1].C; //自定义编码
15.            if (encode1 > encode2){
16.                temp = data[j];
17.                data[j] = data[j + 1];
18.                data[j + 1] = temp;
19.            }
20.        }
21.
22.    for (i = 0; i < n; i++) printf("%d %d %d\n", data[i].A, data[i].B, data[i].C);
23. }
```

适用场景：各字段
类型相同且取值范
围固定

8.2.7 排序

■ 自定义规则排序（方法二：逻辑表达式）

```
1. #include <stdio.h>
2. #define N 100

3. typedef struct { float A; int B; char C; } Data; //A,B,C类型各不相同
4. Data data[N];

5. void main() {
6.     int n, i, j, is_big;
7.     float diff_A; int diff_B; char diff_C;
8.     Data temp;

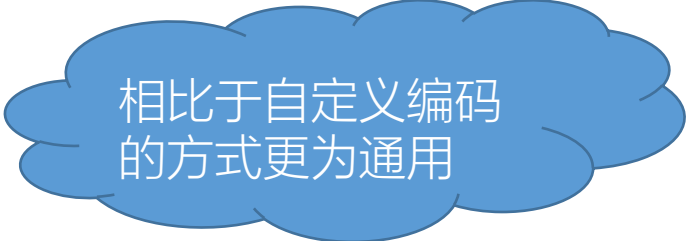
9.     scanf("%d\n", &n);
10.    for (i = 0; i < n; i++) scanf("%f%d%c", &data[i].A, &data[i].B, &data[i].C);

11.    for (i = 0; i < n; i++)
12.        for (j = 0; j < n - i - 1; j++) {
13.            diff_A = data[j].A - data[j+1].A;
14.            diff_B = data[j].B - data[j+1].B;
15.            diff_C = data[j].C - data[j+1].C;

16.            is_big = (diff_A != 0 ? diff_A > 0 : (diff_B != 0 ? diff_B > 0 : diff_C > 0));

17.            if (is_big) { temp = data[j]; data[j] = data[j + 1]; data[j + 1] = temp; }
18.        }

19.    for (i = 0; i < n; i++) printf("%f %d %c\n", data[i].A, data[i].B, data[i].C);
20. }
```



相比于自定义编码的方式更为通用

8.2.8 素数

■ 判断是否是素数

```
1.  #include <stdio.h>

2.  int is_prime(int x) {
3.      int i;
4.      for (i = 2; i <= (int)(sqrt(x)); i++) {
5.          if (x % i == 0) return 0;
6.      }
7.      return 1;
8.  }

9.  void main() {
10.     int x;
11.     scanf("%d", &x);
12.     if (is_prime(x))
13.         printf("%d is a prime\n", i);
14.     else
15.         printf("%d is not a prime\n", i);
16. }
```

■ 筛法求素数

```
1.  #include <stdio.h>
2.  #define N 1001

3.  void main() {
4.      int c, d, k;
5.      //prime[i]代表数字i是否被筛去,
6.      //prime[i]=1代表i被筛去 (即i为合数)
7.      int prime[N];

8.      for (c = 2; c <= N; c=c+1)
9.          prime[c] = 0;    //初始化数组prime

10.     for (d = 2; d <= (int)(sqrt(N)); d++) {
11.         if (prime[d] == 1) continue;
12.         for (k = 2*d; k <= 100; k = k + d)
13.             prime[k] = 1;
14.     }

15.     for (c = 2; c <= N; c=c+1)
16.         if (prime[c] == 0) printf("%d\n", c);
17. }
```


8.3 例题讲解

- YOJ-180: 计算cos的近似值
- YOJ-147: 教室排课
- YOJ-134: 猜数字 -孙浩翔、李甘
- YOJ-483: 数列合并 (简单版) -李佳祎
- YOJ-306: 寻找众数 -卢虹宇
- YOJ-291: 大整数加减 -方言诚、孙浩翔
- YOJ-290: 购物车共同性 -蔡乐宜
- YOJ-296: 班会时间 -王燚
- YOJ-486: 碱基串排序 -郑子姗

YOJ-180: 计算cos的近似值

■ 考点: 循环控制和分析

■ 思路:

- 阶乘, 幂都是典型的使用循环进行计算的操作
- 累加求和也需要使用循环进行
- 需要两重循环
 - 外层循环进行累加
 - 内层循环进行每个分项的计算 (阶乘、幂) 注意事项
- 注意事项
 - +/- 交替出现, 可以用循环计数器%2的值作区分
 - 边界情况: $x^0 = 1$, $0! = 1$
 - sin和cos是周期函数, x可以化约到 $[0, 2\pi]$ 的区间

YOJ-180: 计算cos的近似值

```
#include <stdio.h>
#include <math.h>
#define PI 3.1415926
double get_sin(double, double);
double get_cos(double, double);
void main()
{
    /* get input */
    double x, e;
    scanf("%lf%lf", &x, &e);
    /* scale input x to around 0 */
    while (x > 2 * PI)
        x = x - 2 * PI;
    /* calculation */
    double sin_appro, cos_appro;
    sin_appro = get_sin(x, e);
    cos_appro = get_cos(x, e);
    /* give output */
    printf("%lf\n%lf", sin_appro, cos_appro);
}

/* calculate the approximate value of sin(x) */
double get_sin(double x, double e)
{
    int sign = 1, i = 1, j;
    double item, power, factorial, sin_appro = 0;
    do
    {
        power = 1, factorial = 1;
        /* calculate each item */
        for (j = 1; j <= 2 * i - 1; j++)
        {
            power = power * x;          /* numerator */
            factorial = factorial * j; /* denominator */
```

```
        }
        item = sign * (power / factorial); /* symbol */
        sin_appro = sin_appro + item;      /* add each item to sum */
        sign = sign * (-1);
        i++;
    } while (fabs(item) >= e);
    return sin_appro;
}

/* calculate the approximate value of cos(x) */
double get_cos(double x, double e)
{
    int sign = -1, i = 1, j;
    double item, power, factorial, cos_appro = 1;
    do
    {
        power = 1, factorial = 1;
        for (j = 1; j <= 2 * i; j++)
        {
            power = power * x;
            factorial = factorial * j;
        }
        item = sign * (power / factorial);
        cos_appro = cos_appro + item;
        sign = sign * (-1);
        i++;
    } while (fabs(item) >= e);
    return cos_appro;
}
```

YOJ-147: 教室排课

■ 考点: 枚举和多重循环

■ 思路:

- 枚举法只需要遍历所有可能的情况，并在其中选择出符合要求的即可
- 需要着重考虑如何删选出“符合要求”的枚举情况
- 外部四重循环
 - 每重循环代表一个专业可能要分配的教室
- 内部判断条件
 - 每个专业分配到的教室必须满足人数的限制
 - 不能够有任意两个专业分配到相同的一间教室

YOJ-147: 教室排课

```
#include <stdio.h>

int classrooms(int x);
int main()
{
    /* count of matches */
    int matches = 0;
    /* get input */
    int a, b, c, d;
    scanf("%d %d %d %d", &a, &b, &c, &d);

    /* enumerate all possible plans */
    for (int A = 1; A <= 8; A++)
    {
        for (int B = 1; B <= 8; B++)
        {
            for (int C = 1; C <= 8; C++)
            {
                for (int D = 1; D <= 8; D++)
                {
                    /* check whether each classroom is fit */
                    int c1 = (a <= classrooms(A));
                    int c2 = (b <= classrooms(B));
                    int c3 = (c <= classrooms(C));
                    int c4 = (d <= classrooms(D));

                    /* make sure none of the classrooms are the same*/
                    int e = (A != B) && (A != C) && (A != D) && (C != B) &&
(D != B) && (C != D);

                    /* if all conditions are qualified */
                    if (c1 && c2 && c3 && c4 && e)
                    {
                        printf("%d %d %d %d\n", A, B, C, D);

```

```
matches++;
                    }
                }
            }
        }
    }

    /* if there are no qualified plans */
    if (matches == 0)
        printf("-1");

    return 0;
}

int classrooms(int x)
{
    int values[] = {120, 40, 85, 50, 100, 140, 70, 100};
    if (x >= 1 && x <= 8)
        return values[x - 1];
}
```



#134 猜数字

- 从已知条件推理得到满足条件的数比较复杂 => 考虑枚举数，看是否满足条件
- 为了将四位逐位比较而不是再做拆分，把四位分开枚举

```
for(d[0] = 0; d[0] <= 9; d[0] ++)
```

```
    for(d[1] = 0; d[1] <= 9; d[1] ++)
```

```
        for(d[2] = 0; d[2] <= 9; d[2] ++)
```

```
            for(d[3] = 0; d[3] <= 9; d[3] ++)
```

对于这个数 $d[0]d[1]d[2]d[3]$ ，我们需要检查是否满足所有猜测
满足猜测需要两个方面：

(1) 猜对的数字个数相同 (2) 猜对的位置和数字相同

(2) 很好判断，只要判断 $d[k] == guess[i][k]$

如何判断 (1) ？

我们首先要知道，题目所说的“猜对数字的个数”究竟指什么个数
研究样例：

3585

4815 2 1

5716 1 0

7842 1 0

4901 0 0

8585 3 3 这里不是 4，也不是 2

8555 3 2 这里不是 4，也不是 2（出现了 3 个 5，猜多了）

如果我们设 `num_exist[i]` 表示 `i` 是否出现过，读入时令 `num_exist[i] = 1`，则
对 8585，`num_exist[5]=num_exist[8]=1`，会出现错误

显然数字出现的次数也要被引入

我们不再使用 `num_exist[guess[i][k]] = 1`

我们使用 `num_exist[guess[i][k]] ++`

每次检查时，令 `num_exist --`，如果还没减到 0，则是有效猜测，否则无效

```
1  #include <stdio.h>
2
3  int n, guess[15][7], d[7], ans;
4
5  int main() {
6      scanf( format: "%d", &n);
7      //input the guess information
8      for (int i = 1; i <= n; i++)
9          scanf( format: "%1d%1d%1d%1d %d%d", &guess[i][1], &guess[i][2], &guess[i][3], &guess[i][4], &guess[i][5], &guess[i][6])
10     for (d[1] = 0; d[1] <= 9; d[1]++)
11         for (d[2] = 0; d[2] <= 9; d[2]++)
12             for (d[3] = 0; d[3] <= 9; d[3]++)
13                 for (d[4] = 0; d[4] <= 9; d[4]++) {
14                     int flag = 0; //if success, flag will keep 0
15                     for (int i = 1; i <= n; i++) //check the rule i
16                     {
17                         int num_exist[12] = {0};
18                         num_exist[d[1]]++;
19                         num_exist[d[2]]++;
20                         num_exist[d[3]]++;
21                         num_exist[d[4]]++;
22                         //now we want to check if d[1]d[2]d[3]d[4] satisfies the condition
23                         int num_cnt = 0; //the count of correct numbers
24                         int pos_cnt = 0; //the count of correct number and position
```

```
14     int flag = 0; //if success, flag will keep 0
15     for (int i = 1; i <= n; i++) //check the rule i
16     {
17         int num_exist[12] = {0};
18         num_exist[d[1]]++;
19         num_exist[d[2]]++;
20         num_exist[d[3]]++;
21         num_exist[d[4]]++;
22         //now we want to check if d[1]d[2]d[3]d[4] satisfies the condition
23         int num_cnt = 0; //the count of correct numbers
24         int pos_cnt = 0; //the count of correct number and position
25         for (int k = 1; k <= 4; k++) {
26             int res = guess[i][k];
27             if (num_exist[res]) {
28                 num_cnt++;
29                 num_exist[res]--;
30             }
31             pos_cnt += (res == d[k]);
32         }
33         if (num_cnt != guess[i][5] || pos_cnt != guess[i][6]) {
34             flag = 1;
35             break;
36         }
37     }
38     if (flag == 0) {
39         if (ans == 0) {
40             ans = d[1] * 1000 + d[2] * 100 + d[3] * 10 + d[4];
```

```
36         }
37     }
38     if (flag == 0) {
39         if (ans == 0) {
40             ans = d[1] * 1000 + d[2] * 100 + d[3] * 10 + d[4];
41         } else {
42             printf(format: "Not sure");
43             return 0;
44         }
45     }
46 }
47 if (!ans) printf(format: "Not sure");
48 else printf(format: "%d", ans);
49 return 0;
50 }
```

```

1  #include <stdio.h>
2
3  int n, gs[12][7], d[7], ans, ncnt, pcnt, tg, fg, i;
4
5  int main() {
6      scanf( format: "%d", &n);
7      for (i = 1; i <= n; i++)
8          scanf( format: "%1d%1d%1d%1d %d%d", &gs[i][1], &gs[i][2], &gs[i][3], &gs[i][4], &gs[i][5], &gs[i][6]);
9      for (d[1] = 0; d[1] <= 9; d[1]++)
10         for (d[2] = 0; d[2] <= 9; d[2]++)
11             for (d[3] = 0; d[3] <= 9; d[3]++)
12                 for (d[4] = 0; d[4] <= 9; d[4]++) {
13                     for (i = 1, fg = 0; i <= n && !fg; i++, ncnt = pcnt = 0) {
14                         int exist[12] = {0};
15                         for (int l = 1; l <= 4; l++) exist[d[l]]++;
16                         for (int k = 1; k <= 4; pcnt += (gs[i][k] == d[k]), k++)
17                             (exist[gs[i][k]]--) ? ncnt++ : exist[gs[i][k]]++;
18                         fg = (ncnt != gs[i][5] || pcnt != gs[i][6]);
19                     }
20                     if (!fg) ans ? tg = 1 : ans = d[1] * 1000 + d[2] * 100 + d[3] * 10 + d[4];
21                 }
22      (ans && !tg) ? printf( format: "%d", ans) : printf( format: "Not sure");
23      return 0;
24  }

```

YOJ-134 猜数字

每猜一个数，计算机都会告诉玩家猜对几个数字，其中有几个数字在正确的位置上。

比如计算机随机产生的数字为1122。如果玩家猜1234,因为1,2这两个数字同时存在于这两个数中，而且1在这两个数中的位置是相同的，所以计算机会告诉玩家猜对了2个数字，其中一个在正确的位置。如果玩家猜1111,那么计算机会告诉他猜对2个数字，有2个在正确的位置。

现在给你一段猜数字的过程，你的任务是根据这段对话确定这个四位数是什么。

先弄清楚任务：找到计算机生成的四位数是什么

然后用大白话描述思路：每给出一个“条件”，剔除不符合要求的数

任务拆解：（序号不表示执行顺序，只表示功能的划分）

- 1.判断一个数是否符合一个条件
 - （1）给定玩家猜的数和计算机生成的数，计算猜对几个数字
 - （2）给定玩家猜的数和计算机生成的数，计算正确的位置
- 2.执行这个“剔除不符合要求的数”的过程
（自顶向下，逐步细化）

计算正确的位置

```
int cal_correct_positons(int target, int guess)
{
    int target_numbers[4]; // 计算机生成的数的四位分别是
    int guess_numbers[4];  // 用户猜的数的四位分别是
    for (int i = 0; i < 4; i++) {
        target_numbers[i] = target % 10;
        target /= 10;
    }
    for (int i = 0; i < 4; i++) {
        guess_numbers[i] = guess % 10;
        guess /= 10;
    }
    int counter = 0; // 统计相同的位置数
    for (int i = 0; i < 4; i++) {
        if (target_numbers[i] == guess_numbers[i])
            counter++;
    }
    return counter;
}
```

计算猜对几个数字

```
int cal_correct_numbers(int target, int guess)
{
    int target_numbers[10] = {0}; // 数字 i 在计算机生成的数中出现几次
    int guess_numbers[10] = {0};  // 数字 i 在用户猜的数中出现几次
    for (int i = 0; i < 4; i++) {
        target_numbers[target % 10]++;
        target /= 10;
    }
    for (int i = 0; i < 4; i++) {
        guess_numbers[guess % 10]++;
        guess /= 10;
    }
    int counter = 0; // 统计重合的数字的总数
    for (int i = 0; i < 10; i++) {
        counter += MIN(target_numbers[i], guess_numbers[i]);
    }
    return counter;
}
```

`#define MIN(A, B) ((A) > (B)) ? (B) : (A)`

执行这个“剔除不符合要求的数”的过程

```
int main()
{
    init();
    bool posible[10000];
    int N;
    scanf("%d", &N);
    for (int i = 0; i < N; i++) {
        int guess, correct_numbers, correct_positions;
        scanf("%d%d%d", &guess, &correct_numbers,
            &correct_positions);
        // 淘汰不满足条件的数
        select(guess, correct_numbers, correct_positions, correct_positions)
    }
    {
        for (int i = 0; i < 10000; i++) {
            int unique_number;
            if (cal_correct_numbers(i, guess) != correct_numbers ||
                cal_correct_positions(i, guess) != correct_positions)
                continue;
            if (posible[i]) {
                unique_number = i;
                count++;
            }
        }
        if (count == 1)
            printf("%d", unique_number);
        else
            printf("Not sure");
        return 0;
    }
}
```


#483 数字合并（简单版）

by李佳祎

思索一下：

已知：一个数 (sum)

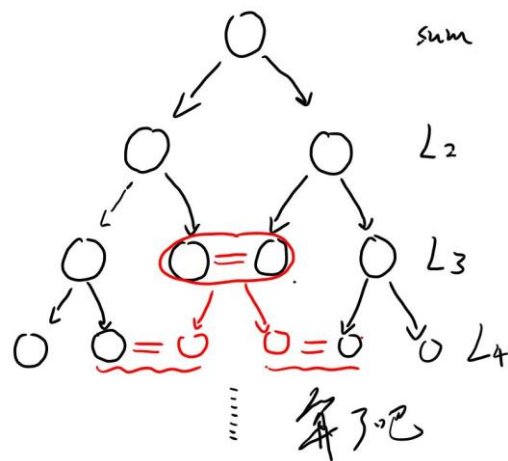
想求：5个数的排列

正推(拆数)：//从sum入手进行操作

由sum, 拆成2个数，再把2个数拆成3个数...一直：

略显庞杂

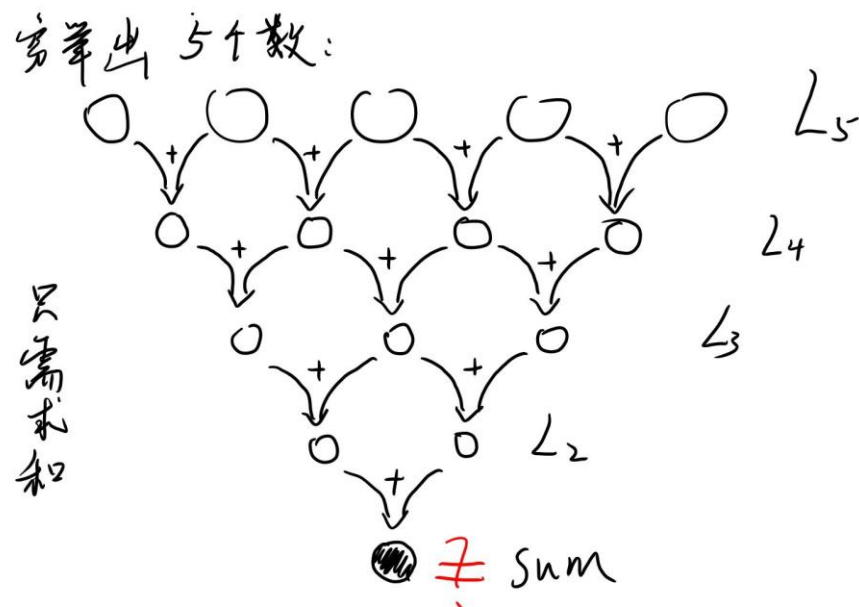
(不知可不可行？)



<正难则反> 及时止损，不必撞南墙

反推(合数)：//穷举出5数排列，最后和sum核对

妙哉！



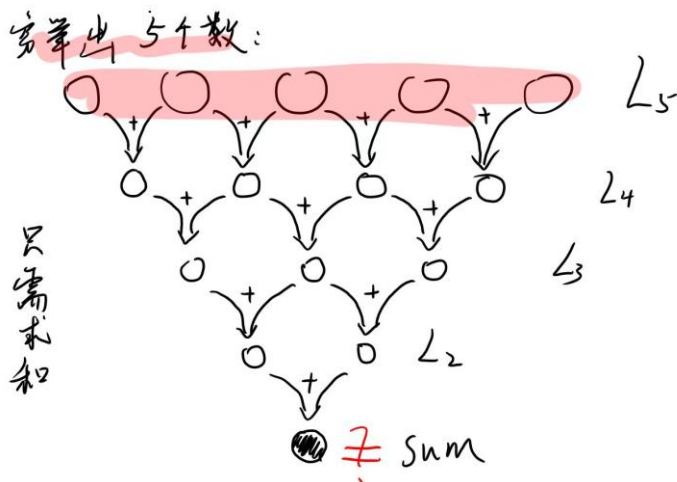
动手尝试:

```

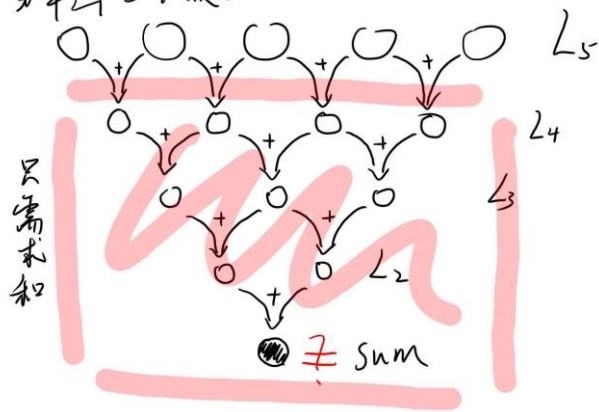
1  #include <stdio.h>
2  int main()
3  {
4      int L5[5], L4[4], L3[3], L2[2], sum;
5      // Lx数组存放x个数的序列
6      // sum是输入的最终数字
7      scanf("%d", &sum);
8      // 5层循环嵌套
9      for (L5[0] = 1; L5[0] < 6; L5[0]++)
10     {
11         for (L5[1] = 1; L5[1] < 6; L5[1]++)
12         {
13             for (L5[2] = 1; L5[2] < 6; L5[2]++)
14             {
15                 for (L5[3] = 1; L5[3] < 6; L5[3]++)
16                 {
17                     for (L5[4] = 1; L5[4] < 6; L5[4]++)
18                     {
19                         if (L5[0] != L5[1] && L5[0] != L5[2] && L5[0] != L5[3] && L5[0] != L5[4]
20                             && L5[1] != L5[2] && L5[1] != L5[3] && L5[1] != L5[4]
21                             && L5[2] != L5[3] && L5[2] != L5[4]
22                             && L5[3] != L5[4])
23                             { // 排除序列中出现重复数字的情况

```

只需要和



穷举出 5 个数:



```
{ // 排除序列中出现重复数字的情况
```

```
// 演算过程
```

```
L4[0] = L5[0] + L5[1];
```

```
L4[1] = L5[2] + L5[1];
```

```
L4[2] = L5[3] + L5[2];
```

```
L4[3] = L5[3] + L5[4]; // 由枚举出来的L5序列 按题目要求 相邻求和 得到L4
```

```
L3[0] = L4[0] + L4[1];
```

```
L3[1] = L4[2] + L4[1];
```

```
L3[2] = L4[2] + L4[3]; // 由L4序列 相邻求和 得到L3
```

```
L2[0] = L3[0] + L3[1];
```

```
L2[1] = L3[2] + L3[1]; // 由L3序列 相邻求和 得到L2
```

```
if (sum == L2[0] + L2[1]) // 最终核对---这种枚举情况得出的数字等于sum吗?
```

```
// 即: L2中两个数加起来 是否等于sum (输入进来的那个数)
```

```
{
```

```
    printf("%d %d %d %d %d\n", L5[0], L5[1], L5[2], L5[3], L5[4]);
```

```
    return 0;
```

```
    /*
```

```
    题目要求输出 “字典序最小的那一个”
```

```
    所以一旦枚举出第一个符合要求L5,那就打印出来, 并马上终止程序
```

```
    (后面可能还会有符合要求的L5 但是字典序就不是最小的了)
```

```
    */
```

```
}
```

输出要求——“字典序”

像字典一样 (strcmp)

字符串比较函数

strcmp(字符串1, 字符串2)

```
idx++;  
return str1[idx] - str2[idx];  
}
```

eg:

abandon

abandoz

abbaaaa

acaaaaa

b.....

c.....

....

z.....

- 作用：比较字符串1和字符串2 (最异元素的ASCII码)
- 字符串比较的规则是：将两个字符串自左至右逐个字符相比 (按ASCII码值大小比较)，直到出现不同的字符或遇到'\0'为止。
 - (1) 如全部字符相同，则认为两个字符串相等；
 - (2) 若出现不相同的字符，则以第1对不相同的字符的比较结果为准。

for循环嵌套 暗中解决 字典序问题

```
for(第一位数字)
{
    for(第二位数字)
    {
        for(第三位)
        {
            for(第四位)
            {
                for(第五位)
                {枚举完成, 开始判断}
            }
        }
    }
}
```

for嵌套运行顺序:

外层循环变量固定住, 先把内层循环的每种情况全走完,
才回到外层循环,
来到外层循环的下一情况

越靠内的循环, 走的次数越多

对应“字典序”:

把靠前的量 (eg: **a**bandon) 放外层循环
把靠后的量 放内层循环

实现在 靠前的量 一致 的情况下, 先对最后的量进行比较,

(eg: abandon
abando**z**
从后往前。。。。)

这样, 枚举出来的第一个L5结果, 就是字典序最小的结果!

1.1.1 YOJ-306:寻找众数

■ 思路:

- 使用数组储存数字出现次数，该数字为数组的下标。
- 利用flag检验数字序列中是否有众数
- 最后利用条件表达式筛选出数值最大的众数

1.1.1 YOJ-306:寻找众数

```
#include <stdio.h>
int main()
{
    int n;//数字序列长度
    scanf("%d", &n);
    int num[n], count[10000] = {0}, flag = 0;//count对数字计算出出现次数计
数
    for (int k = 0; k < n; k++)
    {
        scanf("%d", &num[k]);
        count[num[k]]++;
    }
    for (int v = 0; v < 10000; v++)
    {
        for (int x = v + 1; x < 10000; x++)
        {
            if (count[v] != count[x] && count[v] != 0 && count[x] != 0)
            {
                flag = 1;
                break;
            }
        }
    }
    if (flag == 0)
    {
        printf("NO");
    }//保证数字序列中有众数
    else
    {
        int k1 = 0;
```

```
        for (int t = k1 + 1; t < 10000; t++)
        {
            k1 = (count[k1] > count[t]) ? k1 : t;
        }
        printf("%d %d", k1, count[k1]);
    }//取出现次数最多的数，需要注意的是当两数皆为众数，由于大的数后被遍历到，
    所以自动取较大的数
}
```

yoj-291大整数加减（高精度）

首先我们要确定做这道题的要点：

1.如何字符串来表示输入的数字及计算加法？

eg:

数字291→字符“291”→按位排列 2 9 1

数字3650→字符“365”→按位排列 3 6 5 0

按位相加 3 8 14 1

进位 3 9 4 1

- 用代码表示即为：

// 加法函数

```
void add(char num1[], char num2[],  
char result[])
```

```
{  
    re(num1);  
    re(num2);  
    int len1 = strlen(num1);  
    int len2 = strlen(num2);  
    int max = (len1 > len2) ? len1 : len2;  
    int carry = 0; // 进位符号  
    for (int i = max - 1, j = len1 - 1, k =  
        len2 - 1; i >= 0; i--, j--, k--) //从末  
        位开始运算
```

确保将数据按位输入正确

```
{  
    int n1 = (j >= 0) ? num1[j] - '0' : 0;  
    int n2 = (k >= 0) ? num2[k] - '0' : 0;  
    int sum = n1 + n2 + carry;  
    carry = sum / 10;  
    result[i] = (sum % 10) + '0';  
    if (i == 0 && carry == 1)  
    {  
        移位函数: memmove(result+1, result, max);  
        result[0] = '1';  
    } //避免carry==1而没有进位  
}
```

要是没有这部分：999+999=998
(1998少了最前位)

减法/负号

2.如何运行减法?

3.怎么解决负号带来的问题?

4.小数减大数该如何输出负号?

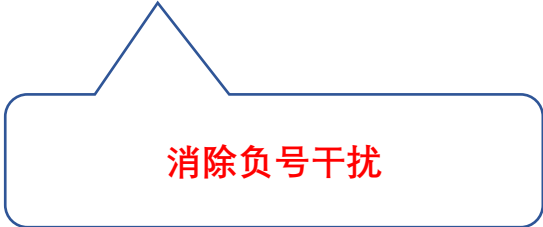
将所有问题转换为**大数减小数**上

// 比大小函数

```
int compare(char num1[], char num2[])
{
    if (strlen(num1) < strlen(num2)) // 比较长度
        return 0;
    else if (strlen(num1) > strlen(num2))
        return 1;
    else
    {
        for (int i = 0; i <= strlen(num1); i--) // 若位
            数相同，逐次比较
            if (num1[i] < num2[i])
                return 0;
    }
    return 1;
}
```

// 绝对值函数

```
void re(char num[])
{
    if (num[0] == '-')
    {
        memmove(num, num + 1, strlen(num));
    }
}
```



消除负号干扰

// 减法函数

void subtract(char num1[], char num2[], char result[])

```
{
    int len1 = strlen(num1); // 被减数
    int len2 = strlen(num2); // 减数
    int borrow = 0; // 借位符号
    for (int j = len1 - 1, k = len2 - 1; j >= 0; j--, k--)
    {
        int n1 = (j >= 0) ? num1[j] - '0' : 0;
        int n2 = (k >= 0) ? num2[k] - '0' : 0;
        int diff = n1 - n2 - borrow;
        if (diff < 0)
        {
            diff += 10;
            borrow = 1;
        }
    }
}
```

else

```
{
    borrow = 0;
}
result[j] = diff + '0';
}

int p = 0;
while (result[p] == '0')
{
    p++;
}

memmove(result, result + p, len1 - p);
result[len1 - p] = '\0'; // 移位使首几位不为0
}
```

要是没有这部分：1000-999=0001
(正确答案应该只有“1”)

以下是主函数部分

//准备工作

```
int main()
```

```
{
```

```
    char s;           // 加减号判断
```

```
    char num1[2001] = {0}; // 特大整数1
```

```
    char num2[2001] = {0}; // 特大整数2
```

```
    char result[2002] = {0}; // 输出数
```

```
    scanf("%c", &s);
```

```
    scanf("%s", num1);
```

```
    scanf("%s", num2);
```

// “+”的转换

if (s == '+') // 这一部分的+1也可以用re函数来实现

这里运用了一些指针的性质，其实就是把“-”移掉
自定义re函数完全可以解决

```
    add(num1 + 1, num2 + 1, result);
    printf("-%s\n", result);
}
else if (num1[0] == '-')
{
    if (compare(num1 + 1, num2) == 0)
    {
        subtract(num2, num1 + 1, result);
        printf("%s\n", result);
    }
    else
    {
        subtract(num1 + 1, num2, result);
        printf("-%s\n", result);
    }
}
```

```
else if (num2[0] == '-')
{
    if (compare(num1, num2 + 1) == 0)
    {
        subtract(num2 + 1, num1, result);
        printf("-%s\n", result);
    }
    else
    {
        subtract(num1, num2 + 1, result);
        printf("%s\n", result);
    }
}
else
{
    add(num1, num2, result);
    printf("%s\n", result);
}
}
```


// “-”的转换

```
else if (s == '-')
{
    if (num1[0] == '-' && num2[0] == '-')
    {
        if (compare(num1 + 1, num2 + 1) == 0)
        {
            subtract(num2 + 1, num1 + 1, result);
            printf("%s\n", result);
        }
        else
        {
            subtract(num1 + 1, num2 + 1, result);
            printf("-%s\n", result);
        }
    }
    else if (num1[0] == '-')
    {
        add(num1 + 1, num2, result);
        printf("-%s\n", result);
    }
}
```

```
else if (num2[0] == '-')
{
    add(num1, num2 + 1, result);
    printf("%s", result);
}
else
{
    if (compare(num1, num2) == 0)
    {
        subtract(num2, num1, result);
        printf("-%s\n", result);
    }
    else
    {
        subtract(num1, num2, result);
        printf("%s\n", result);
    }
}
return 0;
}
```

大整数加减法（高精度算法）：

解决的思路是自然的，我们只要能够实现 $a-b$ 和 $a+b$ ，其中 $a \geq b \geq 0$ 于是我们考虑竖式加减法，即将 a, b 右对齐，从右开始逐位相加相减
具体地说，需要实现进位和借位

如何实现？

（1）如何实现右对齐与从右开始计算？

一种思路是（我的思路）将数字倒着从第一个开始存，从第一个开始算

1234	4321
+ 345	543
1579	9751

（2）如何实现进位？定义进（退）位变量 pre

如果加出去了 10 或者减到了负数，就进位或退位：注意加法最多进 1

（3）如何保证 $a > b$ ：实现比较和交换函数

(4) 如何实现符号规整? (自行分类推导)

(5) 注意的小点

- a. $900+900$ 是 1800, 最后需要再考虑一次进位
- b. 如果 $1820-1800$, 不要输出 0020: 前导 0 问题
- c. 如果是 0, 不要输出 -0
- d. 实际上, 加减法本质一样, 可以简化程序

```

1  #include <stdio.h>
2
3  #define maxL 2010
4
5  int num_1[maxL + 1], num_2[maxL + 1];
6  → int output[maxL + 1], cnt;
7
8  void swap(int *x, int *y) {
9      int tmp = *x;
10     *x = *y;
11     *y = tmp;
12 }
13
14 void swap_num(int num1[], int num2[]) {
15     for (int i = 1; i <= num2[0]; i++)
16         swap(&num1[i], &num2[i]);
17     swap(&num1[0], &num2[0]);
18 }
19
20 int compare(int num1[], int num2[]) {
21     if (num1[0] > num2[0]) return 1;
22     if (num1[0] < num2[0]) return -1;
23     for (int i = num1[0]; i >= 1; i--) {
24         if (num1[i] > num2[i]) return 1;
25         if (num1[i] < num2[i]) return -1;
26     }
27     return 0;
28 }

```

```

30 int input_num(int num[]) {
31     char current = getchar();
32     int tmp[maxL + 1] = {0};
33     int positive_check = 1, num_len = 0;
34     if (current == '-') {
35         positive_check = -1;
36         current = getchar();
37     }
38     while ('0' <= current && current <= '9') {
39         tmp[++num_len] = current - '0';
40         current = getchar();
41     }
42     for (int i = 1; i <= num_len; i++)
43         num[i] = tmp[num_len - i + 1];
44     num[0] = num_len;
45     return positive_check;
46 }
47
48 void add_or_sub(int num1[], int num2[], int op) { //op=1 add, op=-1 sub
49     int pre = 0;
50     for (int i = 1; i <= num1[0]; i++) {
51         int current = num1[i] + op * num2[i] + pre;
52         pre = (current >= 10 || current < 0) ? op : 0;
53         num1[i] = current - 10 * pre;
54     }
55     if (pre == 1) num1[++num1[0]] = 1;
56 }
57

```

```

58 int print(int num1[]) {
59     int flag = 0;
60     for (int i = num1[0]; i >= 1; i--) {
61         if (num1[i] == 0 && !flag) continue;
62         if (num1[i] != 0 && !flag) flag = 1;
63         output[++cnt] = num1[i];
64     }
65     return flag;
66 }
67
68 int main() {
69     char op = getchar();
70     int all_op = (op == '+') ? 1 : -1;
71     getchar();
72     int num1_positive = input_num(num: num_1);
73     int num2_positive = input_num(num: num_2);
74     if (compare(num1: num_1, num2: num_2) == -1) {
75         swap_num(num1: num_1, num2: num_2);
76         swap(x: &num1_positive, y: &num2_positive);
77         num1_positive = all_op * num1_positive;
78         num2_positive = all_op * num2_positive;
79     }
80     int check = num1_positive * num2_positive * all_op;
81     add_or_sub(num1: num_1, num2: num_2, op: check);
82     int check_all_zero = print(num1: num_1);
83     if (num1_positive == -1 && check_all_zero) printf(format: "-");
84     if (!check_all_zero) printf(format: "0");
85     for (int i = 1; i <= cnt; i++)

```

```

85     for (int i = 1; i <= cnt; i++)
86         printf(format: "%d", output[i]);
87     return 0;
88 }

```

源代码见

<https://send.coderbak.com/-sP2Z7i7GcF/main.c>

290-购物车共同性

电商平台在购物狂欢节期间，从某类顾客中随机抽取 n 个顾客($n > 1$)，调查他们购物车中预选保存的商品类别，以掌握顾客购物偏好的共同性，得到最受欢迎的商品，为拓展市场销量服务。购物车中商品会被分类，假定种类是以整数进行编号，此问题为在 n 个集合中寻找交集的问题。

输入格式

第一行一个整数 n ，表示选取的顾客数。此后有 n 行，分别表示每位顾客所选商品构成，其中第一个数表示该顾客购物车中的不同商品数，其后为空格分隔的整数。调查抽取的顾客数 n 不超过20，每个顾客所选的商品种类不超过100。

提示数组大小

输出格式

若不存在共同偏好，无交集，输出NO；若有，把共同的商品编号从小到大输出，空格分隔。

输出规范：排序

290-购物车共同性

可能用到的板块内容：

1. 二维数组
2. 冒泡/选择排序
3. 循环查找相同数据
4. 退出循环的break条件
5. 没有找到相关数据的输出条件

290-购物车共同性

```
#include <stdio.h>
int main()
{
    int n, flag[300][30] = {0}, flag1 = 0, flag2 = 0, len[30], i, j, k, m, number, result[300] = {0}, count, temp;
    // flag数组: 存放是否有相同商品的数组, 如果有则记为1
    // flag1: 这个物品是否在n个购物车都有
    // flag2: 是否找到了n个都有的数据, 若没有则输出NO
    // len: 每个购物车有几个物品
    // result: 存放输出的商品, 方便从小到大排序
    // count: 有几个这样共同存在的商品
    scanf("%d", &n);
    int a[30][300] = {0};
    for (i = 1; i <= n; i++)
    {
        scanf("%d", &len[i]);
        for (j = 1; j <= len[i]; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
    //输入商品数量和编号
```


290-购物车共同性

```
for (j = 1; j <= len[1]; j++)
{
    for (i = 2; i <= n; i++)
    {
        for (k = 1; k <= len[i]; k++)
        {
            if (a[1][j] == a[i][k])
            {
                flag[j][i] = 1; //以第一行为标准搜索，若找到了，该位置标记为1
            }
        }
    }
    for (m = 2; m <= n; m++)
    {
        flag1 = 0;
        if (flag[j][m] == 0)
        {
            flag1 = 1; //对于某一特定物品只要某一个flag是0就不符合
            break;
        }
    }
}
```

```
    if (flag1 == 0)
    {
        count++;
        result[count] = a[1][j];
        flag2 = 1; //表示找到了至少一组可输出的数据
    }
}
if (flag2 == 0) //没有共同商品的情况
{
    printf("NO");
}
else
{
    for (i = 1; i <= count; i++)
    {
        for (j = 1; j <= count - i; j++)
        {
            if (result[j] > result[j + 1])
            {
                temp = result[j];
                result[j] = result[j + 1];
                result[j + 1] = temp; //对结果按照商品序号进行排序
            }
        }
    }
    for (i = 1; i <= count; i++)
    {
        printf("%d ", result[i]);
    }
}
return 0;
}
```

YOJ-296: 班会时间

- 考点: 数组输入、字符串处理和排序

- 思路:

第一步: 由题, 要统计1.1--7.7共49个数字, 不妨将其先化为整数, 即考虑这49个数字:

- 11-77 (个位十位均从1列举至7)
- 构建一个数组长度为49 用来统计各个课程的数量 但这需要进行一下换算:
- 比如11应该让arr1[0]++; 77应该让arr1[48]++;不难发现, 如果枚举的数字是 $i*10+j$
- 那么数组对应的位置应该是 $(i-1)*7+j-1$

第二步: 按顺序输出

从0开始, 查找相应的课程对应位置, 并输出相应的课程

```
#include D:\2023 Programming I
int main()
{
    int n, k, p, m, a, q, w, order = 0, order1 = 0, enough = 0, flag = 0;
    float lesson;
    int arr1[49] = {0}; // 记录课程出现次数
    int arr2[49] = {0}; // 与数组arr1相对应的课程
    scanf("%d%d", &n, &k);
    for (int i = 1; i <= 7; i++)
    {
        for (int j = 1; j <= 7; j++)
        {
            arr2[order] = i * 10 + j;
            order++;
        }
    }
    for (int i = 0; i < n; i++)
    {
        scanf("%d%d", &m, &p);
        for (int j = 0; j < p; j++)
        {
            scanf("%f", &lesson);
            a = lesson * 10;

            q = a % 10;
            a /= 10; // 个位q
            w = a;   // 十位w
            arr1[(w - 1) * 7 + q - 1]++;
        }
    }
}
```

```

for (int i = 0; i < 2000; i++)
{
    if (flag)
        break;
    for (int j = 0; j < 49; j++)
    {
        if (enough == k)
        {
            flag++;
            break;
        }
        if (arr1[j] == order1)
        {
            printf("%d.%d %d\n", arr2[j] / 10, arr2[j] % 10, order1);
            enough++;
        }
    }
    order1++;
}
return 0;

```

对应位置输出：
 比如0对应1.1
 arr2[]按顺序装有
 11...17
 ...
 71...77

486 碱基串排序

自定义排序向常规数字排序的转化 排序方式变化
则采取一个普适的方法 使
排序方式不变

【问题描述】

脱氧核糖核酸（DNA）是由以下四种碱基组成的双螺旋结构：A(ADENINE腺嘌呤)、T(THYMINE胸腺嘧啶)、G(GUANINE鸟嘌呤)、C(CYTOSINE胞嘧啶)。现给定N个长度相同的碱基串（注意：串由多个有序的碱基组成，每个碱基只能为ACGT中的一个），请你对它们按用户指定的优先次序进行排序。

【输入格式】

第1行为一个正整数N，表示需要排序的碱基串的个数。

第2行为用户指定的排序规则，共包含4个字符，为A、C、G、T四个碱基的一个全排列，表示单个碱基的优先次序，出现的越靠前则越优先。根据单个碱基的优先次序，按照如下规则比较两个碱基串a与b：从a和b的第一个碱基开始比较，如果前者比后者优先，则a比b优先；如果后者比前者优先，则b比a优先；如果二者相等，则比较后一个碱基进行判断，以此类推。

接下来N行，每行一个碱基串。注意每个碱基串长度相同。

【输出格式】

共输出N行，每行一个碱基串，按照用户指定的优先规则排好顺序。

【注意】

对于所有数据，碱基串的个数 $N \leq 1000$ ，每个碱基串的长度 ≤ 20 。

【输入样例1】

```
4
ACGT
T
G
C
A
【输出样例1】
A
C
G
T
```

【样例输入2】

```
5
TGCA
AAT
CAA
AGT
AGG
ACA
【样例输出2】
CAA
AGT
AGG
ACA
AAT
```

```

1  #include <stdio.h>
2  #include <string.h>
3  int main()
4  {
5      int n, i, j, num[1001][21] = {0}, temp, k, m, flag;
6      char a[5], dna[1001][21] = {0};
7      scanf("%d", &n);
8      scanf("%s", a);
9      for (i = 0; i <= n - 1; i++)
10     {
11         scanf("%s", dna[i]);
12     }
13     m = strlen(dna[1]); // 确定输入碱基串的长度
14     for (i = 0; i <= n - 1; i++)
15         for (j = 0; j <= m - 1; j++)
16             {
17                 if (dna[i][j] == a[0])
18                 {
19                     num[i][j] = 1;
20                     continue;
21                 }
22                 if (dna[i][j] == a[1])
23                 {
24                     num[i][j] = 2;
25                     continue;
26                 }
27                 if (dna[i][j] == a[2])
28                 {
29                     num[i][j] = 3;
30                     continue;
31                 }
32                 if (dna[i][j] == a[3])
33                 {
34                     num[i][j] = 4;
35                     continue;
36                 }
37                 else
38                     continue; // 将碱基串字符转为数字存储到数组里 字符比较转为数字比较

```

排序转化：

令指定顺序xxxx中的第一个碱基x1相当于数字1，第二个碱基x2相当于数字2，以此类推

再把待比较的碱基串转为二维数组 即令碱基x1为1，碱基x2为2....

从而将字母的排序变成数字大小比较
数字小的字符串优先

则不管用户指定的排列顺序怎么变化
比较方法和代码实现都一样

输入碱基串长度相同 故先用
strlen求出长度再循环 可减少循
环次数

数组排序过程

```
39     }
40     for (k = 0; k <= n - 1; k++)
41         for (i = 0; i <= n - k - 2; i++)
42         {
43             for (j = 0; j <= m - 1; j++)
44             {
45                 if (num[i][j] > num[i + 1][j])
46                 {
47                     for (j = 0; j <= m - 1; j++)
48                     {
49                         temp = num[i][j];
50                         num[i][j] = num[i + 1][j];
51                         num[i + 1][j] = temp;
52                     }
53                     flag = 1;
54                 }
55                 else if (num[i][j] == num[i + 1][j])
56                 {
57                     continue;
58                 }
59                 else
60                 {
61                     break;
62                 }
63             }
64             if (flag)
65                 break;
66         }
67     } // 排序
```



```

68     for (i = 0; i <= n - 1; i++)
69         for (j = 0; j <= m - 1; j++)
70             {
71                 if (num[i][j] == 1)
72                     {
73                         dna[i][j] = a[0];
74                         continue;
75                     }
76                 else if (num[i][j] == 2)
77                     {
78                         dna[i][j] = a[1];
79                         continue;
80                     }
81                 else if (num[i][j] == 3)
82                     {
83                         dna[i][j] = a[2];
84                         continue;
85                     }
86                 else if (num[i][j] == 4)
87                     {
88                         dna[i][j] = a[3];
89                         continue; // 排序好的数组转回字符串
90                     }
91             }
92     for (i = 0; i <= n - 1; i++)
93         printf("%s\n", dna[i]);
94
95     return 0;
96 }

```

排序好的数组转回字符串